04820KA001 Mono Canada II_Ital_Mod 1019_10

Author: Michou
Created: 10/19/2026 8:06:19 PM
Last Modified: 10/19/2026 8:47:45 PM
Comment: Modified to Connect 2nd Diameter Sens

Sysmac Studio Module Version: 1.60.0.64010

1.PLC

1-7.POUs

1-7-1.Programs

1-7-1-1.TaglMonoTRM

1-7-1-1-1.Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Comment
VAR						
SpdDelta	REAL			False	False	Minimum or maximum speed SpdDeltaerence
SpdDeltaMin	REAL			False	False	Minimum speed difference
SpdDeltaMax	REAL			False	False	Maximum speed SpdDeltaerence
Clock100ms	BOOL			False	False	100 ms clock
MemPos	DINT			False	False	Previous encoder count
DeltaPulse	DINT			False	False	Encoder count difference
Velocity	REAL			False	False	Speed in [mm/s]
Alarm_Array	ARRAY[0..32] OF BOOL			False	False	Alarm
T_PlcOn	TON			False	False	PLC active timeout
ControlVoltage	BOOL			False	False	Auxiliaries OK
StartMotor_Cmd	BOOL			False	False	Motor start command
HubExtractEnable	BOOL			False	False	Hub extraction enable command
Brake_Cmd	BOOL			False	False	Brake command
T_StopVent	TOF			False	False	Fans stop timer
TimStopVen	TIME			False	False	Fans stop timeout
MotorCooling	BOOL			False	False	Fans command
OverRider_Auto	BOOL			False	False	OverRider in automatic
OverRider_Cont	BOOL			False	False	OverRider in continuous
OverRider_Lock	BOOL			False	False	OverRider lock command
TickCnt	INT			False	False	100 ms tick scan counter (25 x 4 ms)
RunWasJog	BOOL			False	False	Run used jog / manual / slow button (2nd ramp)
StopLearnValid	BOOL			False	False	This stop may be learned
SpdBand_Pps	REAL			False	False	Speed band of the trim ramp (pulses/s)
TrimDecel_Pps2	REAL			False	False	Trim ramp decel (pulses/s2)
SpdKnee_Pps	REAL			False	False	Speed where coarse ramp hands over to trim ramp (pulses/s)
OverRider_RaiseUp	BOOL			False	False	OverRider raise command
Pot1_Mem	INT			False	False	Work potentiometer value
Clock10ms	BOOL			False	False	10 ms clock
VelSet_mms	REAL			False	False	Set speed in mm/s
Vfd_SP_Modif	REAL			False	False	Inverter speed modifier
T_PosPresOff	TON			False	False	OverRider photocell OFF timer
T_PosPresOn	TON			False	False	OverRider photocell ON timer
TimPosPresOff	TIME			False	False	OverRider photocell OFF timeout
TimPosPresOn	TIME			False	False	OverRider photocell ON timeout
RollEnd_Active	BOOL			False	False	Roll diameter photocell intercepted
Output_Ratio	REAL			False	False	Analog output ratio
Input_Ratio	REAL			False	False	Analog input ratio
T_StartLow	TON			False	False	Slow start timer
T_UwDiamDebounce	TON			False	False	Roll diameter photocell Debounce timer 500mS

UwDiamReached	BOOL			False	False	Roll diameter reached
Index_A	INT			False	False	
AlarmStrobe	BOOL			False	False	Alarm message display strobe
T_Display	TON			False	False	Alarm display timer
RangeOK	BOOL			False	False	Range reached
Vel_Max_mms	REAL			False	False	Maximum speed in mm/s
Slow_Inhibit	BOOL			False	False	Inhibit slow button
Decel_Ramp	REAL			False	False	Deceleration ramp
Accel_Ramp	REAL			False	False	Acceleration ramp
Pot2_Mem	INT			False	False	Final potentiometer value
Current_mm	REAL			False	False	Current millimetres
EncoderPulse_to_mm	REAL			False	False	mm per encoder pulse
SlowDown	BOOL			False	False	Slow down
Stop	BOOL			False	False	Stop
T_EncoderStill	TON			False	False	(?) Encoder stopped timer
Analog_Res	UINT			True	False	Analog board resolution
Mt_To_mm	REAL			False	False	m/min to mm/s factor
Sec_to_Min	REAL			False	False	mm/s to m/min factor
Pot1_Word	WORD			False	False	(?) Pot 1 raw word
Pot1_WordSW	WORD			False	False	(?) Pot 1 raw word, byte-swapped
Pot2_Word	WORD			False	False	(?) Pot 2 raw word
Pot2_WordSW	WORD			False	False	(?) Pot 2 raw word, byte-swapped
InvRef2	UINT			False	False	(?) Inverter reference 2
InvRefSW	WORD			False	False	(?) Inverter reference, byte-swapped
InvRefWW	WORD			False	False	(?) Inverter reference, word
Init_Ramp	BOOL			True	False	(?) Ramp initialisation
T_CoreMemDelay	TON			False	False	(?) Core value stored timer
SlowByDiam	BOOL			False	False	(?) Slow down by diameter
UwStopByDiam	BOOL			False	False	Stop by diameter
T_Screen_Delay	TON			False	False	(?) Start page timer
T_PulStop	TON			False	False	(?) Stop button timer
Reading_Max	REAL	2000		True	False	Maximum sensor reading (full scale)
DeadZone	REAL	60		True	False	Sensor dead zone
In_UwDiameter	REAL			True	False	UnWinder Diameter Analog input voltage
T_Sample	TON			False	False	100 ms tick timer
Tick	BOOL			False	False	100 ms tick
MmPerPulse	REAL			False	False	mm per encoder pulse
RwDistRaw_mm	REAL			False	False	Rewinder Unfiltered distance (mm)
RwFiltInit	BOOL			False	False	Rewinder Filter initialised
SpdInit	BOOL			False	False	Speed init
EncPrev	DINT			False	False	Encoder at previous tick
SpdAct_Pps	REAL			False	False	Actual Speed (pulses/s)
LearnInit	BOOL			False	False	Learn initialised
LearnWinRunning	BOOL			False	False	Motor ran whole window
LearnWinTicks	INT			False	False	Learn window counter
Encoder_Last	DINT			False	False	Encoder at window start
Diameter_Last	REAL			False	False	Diameter at window start
LearnWin_Pls	DINT			False	False	Pulses in window
LearnWinDiamDelta	REAL			False	False	Diameter change in window

	MatK_Window	REAL			False	False	Material factor this window
	MotorRunMem	BOOL			False	False	Motor start edge memory
	StopMeasActive	BOOL			False	False	Stop measurement running
	StopStartPos	DINT			False	False	Encoder at stop command
	StopStartSpd_Pps	REAL			False	False	Speed at stop command (pulses/s)
	StandstillTicks	INT			False	False	Standstill tick counter
	LearnWinOk	BOOL			False	False	Learning window gave a valid result
	ThickWin_mm	REAL			False	False	Paper thickness this window (mm)
	LearnFailCnt	INT			False	False	Consecutive failed learning windows
	StopLearnOk	BOOL			False	False	Stop measurement gave a valid result
	StopDecelMeas_mm	REAL			False	False	Measured stop decel (mm/s2)
	StopMeas_Pls	REAL			False	False	Measured stop length (pulses)
	StopDecelMeas	REAL			False	False	Measured stop decel (pulses/s2)
	StopLen_Pls	REAL			False	False	Stop length (pulses)
	StopLenFinal_Pls	REAL			False	False	Stop length from slow speed (pulses)
	RwSlowLen_Pls	REAL			False	False	Rewinder Slow-down length (pulses)
	SlowDecel_Pps2	REAL			False	False	Slow-down decel (pulses/s2)
	SpdFinal_Pps	REAL			False	False	Final speed (pulses/s)
	RwRemain_Pls	REAL			False	False	Rewinder Remaining length to target (pulses)
	LearnWinStartPos	DINT			False	False	Rewinder Position Window 0
	LearnWinStartDiam	REAL			False	False	Rewinder Diameter Window 0
	T_ByPassProtOff	TOF			False	False	Delay Off Timer
	UwEnMem	BOOL			False	False	Edge memory UwStopEnable
	RwEnMem	BOOL			False	False	Edge memory RwEnable
	SpdRef_Pps	REAL			False	False	Reference speed MAX (actual,setpoint) (pulses/s)
	SpdWork_Pps	REAL			False	False	Work speed setpoint (pulses/s)
	SlowLenBase_Pls	REAL			False	False	Slow-down length without margin (pulses)
	UwSlowLen_Pls	REAL			False	False	Unwinder slow-down length (pulses)
	UwMarginUsed_m	REAL			False	False	Unwinder margin at slow speed (m)
	SqrtArg	REAL			False	False	Square root argument
	AutoSlowCmd	BOOL			False	False	Automatic Slow Command
	AutoStopCmd	BOOL			False	False	Automatic Stop Command
	RwLastValidDiam	REAL			False	False	Rewinder Last valid diameter (mm)
	RwRemainMax_Pls	REAL			False	False	Rewinder Remaining length to max diameter (pulses)
	UnWinder_Empty	BOOL			False	False	UnWinder Empty
	StopFailCnt	INT			False	False	Consecutive failed stop measurements
VAR_EXTERNAL							
	VelSet	REAL				False	Set speed
	OverSpeed	BOOL				False	Speed too high
	SpdDelta_Percen	REAL				False	Speed difference percentage

t						
Encoder_Pos	DINT				False	Encoder counter
Velocity_M_min	REAL				False	Speed in m/min
Current_Metre	REAL				False	Current metres
I_RsetCnt	BOOL				False	Metre counter reset button
UnderSpeed	BOOL				False	Speed too low
I_ControlVoltage	BOOL				False	Auxiliaries ON
I_OverRider_PH	BOOL				False	Roll diameter photocell
O_MotorStart	BOOL				False	Motor start
I_OR_Position_Sens	BOOL				False	Rider roll position sensor
I_OR_Pressure_Sens	BOOL				False	Rider roll pressure switch
I_OR_Auto_OS	BOOL				False	Rider roll in automatic
I_OR_Cont_OS	BOOL				False	Rider roll in continuous
I_Start_PB	BOOL				False	Start button
I_SafeGuard_Closed	BOOL				False	Side doors closed
I_Motor_Thermal	BOOL				False	Motor thermal switch
I_FanTemp_Ok	BOOL				False	Motor (fan) thermal OK
I_Stop_PB	BOOL				False	Stop button
I_Hub_Position	BOOL				False	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL				False	Front guard closed
RwMaxSlowFallback	REAL				False	Rewinder max: slow-down diameter while K not learned (mm)
RwMaxSlowAtDiam	REAL				False	Rewinder max: slow-down starts at diameter (mm)
O_Alarm_Array	BOOL				False	Alarm lamp
O_RwUnload	BOOL				False	Unloaders enable - opening
I_Prot_ByPass	BOOL				False	Guards-bypassed selector switch
O_Prot_ByPass_Lamp	BOOL				False	Guards bypassed lamp
O_Brake	BOOL				False	Brake command
Fan_Timeout	REAL				False	Fans stop timeout (from panel)
O_CoolingFan	BOOL				False	Brake and motor fans
O_OR_Auto_Lamp	BOOL				False	Rider roll auto/continuous lamp
O_OverRider_Up	BOOL				False	Raise rider roll
O_OverRider_Hold	BOOL				False	Lock rider roll
Fix10Vdc	INT				False	Fixed 10 Vdc generator
InvRef	INT				False	Inverter speed reference
Encoder_On	BOOL				False	Encoder enable
Encoder_Reset	BOOL				False	Encoder reset
OverRider_TimeOFF	REAL				False	Rider roll photocell OFF timeout (panel)
OverRider_TimeON	REAL				False	Rider roll photocell ON timeout (panel)
VelMax	REAL				False	Maximum speed
I_LowSpd_OS	BOOL				False	Slow speed button
O_Motor_Lamp	BOOL				False	Motor ON lamp
AlarmOn	BOOL				False	Alarm active
BuzzerOn	BOOL				False	Panel buzzer command
I_VFD_Alarm	BOOL				False	Inverter in alarm
Msg	INT				False	Message
Index_B	INT				False	Active alarm index

MotStop	BOOL				False	Motor stopped
Clock1s	BOOL				False	1 s clock
ManMode	BOOL				False	Manual mode
Speed_Man_SP	INT				False	% speed in manual
Counter_Reset	BOOL				False	Panel counter reset button
I_FrontGuard_Open	BOOL				False	Front guard open
RwUnload_Enable	BOOL				False	Unloaders enable - closing
JogSpd_SP	REAL				False	Speed in manual
O_AccDec2	BOOL				False	Second acc/dec ramp
AccelTrim	REAL				False	Acceleration coefficient in run
DecelTrim	REAL				False	Deceleration coefficient in run
DecelCoarse	REAL				False	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL				False	Deceleration coefficient at start (orig. comment; name suggests accel)
Speed_Display	REAL				False	Set speed (displayed)
O_AcousticSignal	BOOL				False	Horn / acoustic signal
Pot1_Work_Velocity	INT				False	Potentiometer 1 value - Work speed -
O_FrontGuard_ByPass	BOOL				False	Front guard bypass
Pot2_Slow_Velocity	INT				False	Potentiometer 2 value - Final speed -
LowSpd_Display	REAL				False	Slow-down speed (displayed)
Encoder_Pulse	INT				False	Encoder pulses
LowSpd_Man_SP	INT				False	% slow-down speed in manual
UwSlowMargin_m	REAL				False	Slow-down metres
UwStop_At_Meter	REAL				False	Total metres
I_AutoStop_OS	BOOL				False	Stop on metre counter
I_RollEnd_SlowDown	BOOL				False	Slow down at end of roll
I_SlowSens_PH	BOOL				False	Slow-down photocell
I_WebBreak_Monitor	BOOL				False	Paper break monitoring ON
I_WebBreak_PH	BOOL				False	Paper break photocell
I_Vacuum_OK	BOOL				False	Extractor fan thermal OK
CountReset_Req	BOOL				False	Metre counter reset request
Psw	INT				False	Password
I_RsetAlarm	BOOL				False	Alarm reset
Fix10Vdc_1	UINT				False	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL				False	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL				False	Panel alarms
SafeGuard_ByPass	BOOL				False	Guards bypassed
FrontGuard_Closed	BOOL				False	Front guard closed
Core_Measure	BOOL				False	(?) Stored core value
UwDiam_Measure	REAL				False	Measured distance
UwCoreDistance	REAL				False	(?) Core measurement

AI_Diam_Unwinder	INT				False	Analog value - parent roll measurement
UwCoreDiameter	REAL				False	(?) Shaft diameter
Measure_ON	BOOL				False	(?) Measurement enabled
UwDiam_Act	REAL				False	(?) Measured roll diameter
UwDiamStop	REAL				False	(?) Stop diameter
UwSlowDown_Diam	REAL				False	(?) Slow-down diameter
Alarm_Reset	BOOL				False	(?) Alarm reset from panel
HMI_Screen	ARRAY[0..1] OF DINT				False	HMI panel page
Psw22	BOOL				False	(?) Password 22
Psw44	BOOL				False	(?) Password 44
I_Roll_OK	BOOL				False	(?) Roll OK
I_RollEnd_PH	BOOL				False	(?) End-of-roll photocell
AI_Diam_Rewinder	INT				False	Rewinder DT35 raw - I/O Map Unit2 Ch2
RwStopEnable	BOOL				False	Rewinder diameter stop enabled
RwTargetDiam	REAL				False	Rewinder final diameter (mm)
RwUseSlow	BOOL				False	Slow down before target
RwSlowMargin_m	REAL				False	Metres at slow speed before stop
RwSlowDecel	REAL				False	Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL				False	Reset requests below target - this (mm)
RwDist0V	REAL				False	DT35 distance at 0 V (mm)
RwDist10V	REAL				False	DT35 distance at 10 V (mm)
RwRawMin	INT				False	Raw below = sensor fault
RwRawMax	INT				False	Raw above = sensor fault
RwStopOnFault	BOOL				False	Stop on sensor fault
RwAxisDist	REAL				False	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL				False	Distance filter per 100 ms (0..1)
RwWinTicks	INT				False	Material learn window (100 = 10 s)
RwMinPulses	DINT				False	Min pulses in window to learn
RwLearn_K	REAL				False	Material factor filter weight
RwMat_K	REAL				False	Learned material factor
RwLearnStop	BOOL				False	Learn stop decel on every stop
RwMinLearnSpd	REAL				False	Min speed to learn a stop (m/min)
RwStopDecel	REAL				False	Learned stop decel (pulses/s2)
RwTreact	REAL				False	Stop reaction time (s)
RwTstopDef	REAL				False	Stop time until decel learned (s)
RwTrim	REAL				False	Stop length trim (>1 = earlier)
RwDist	REAL				False	Filtered distance (mm)
RwDiam	REAL				False	Rewinder diameter (mm)
RwSpeed	REAL				False	Line speed from encoder (m/min)
RwRemain_m	REAL				False	Metres to target diameter
RwStopLen_m	REAL				False	Stopping length now (m)
RwSlowLen_m	REAL				False	Slow-down length now (m)
RwStopReq	BOOL				False	Rewinder diameter STOP request
RwSlowReq	BOOL				False	Rewinder diameter SLOW request
RwSensFault	BOOL				False	Rewinder DT35 sensor fault
UwStopEnable	BOOL				False	Unwinder metre-counter stop ON (HMI)
SlowAtDisp	REAL				False	HMI: slow-down starts at (m or

						mm, see SlowAtMode)
SlowAtMode	INT				False	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL				False	HMI: show slow-down display
UwSlow_at_Metre	REAL				False	Unwinder: slow-down starts at (m)
UwSlowReq	BOOL				False	Unwinder metre slow-down request (latched)
RwSlowAtDiam	REAL				False	Rewinder: slow-down starts at diameter (mm)
RwStopAtDiam	REAL				False	Rewinder: stop command at diameter (mm)
RwMaxDiam	REAL				False	Rewinder max diameter protection (mm)
RwMaxStop	BOOL				False	Rewinder max diameter STOP (latched)
RwMaxSlow	BOOL				False	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL				False	
RwThickMin_mm	REAL				False	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL				False	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL				False	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL				False	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT				False	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT				False	Learning: failed stops before alarm
RwLearnFault	BOOL				False	Rewinder learning fault (alarm 16)
RwThick_mm	REAL				False	Learned paper thickness (mm)

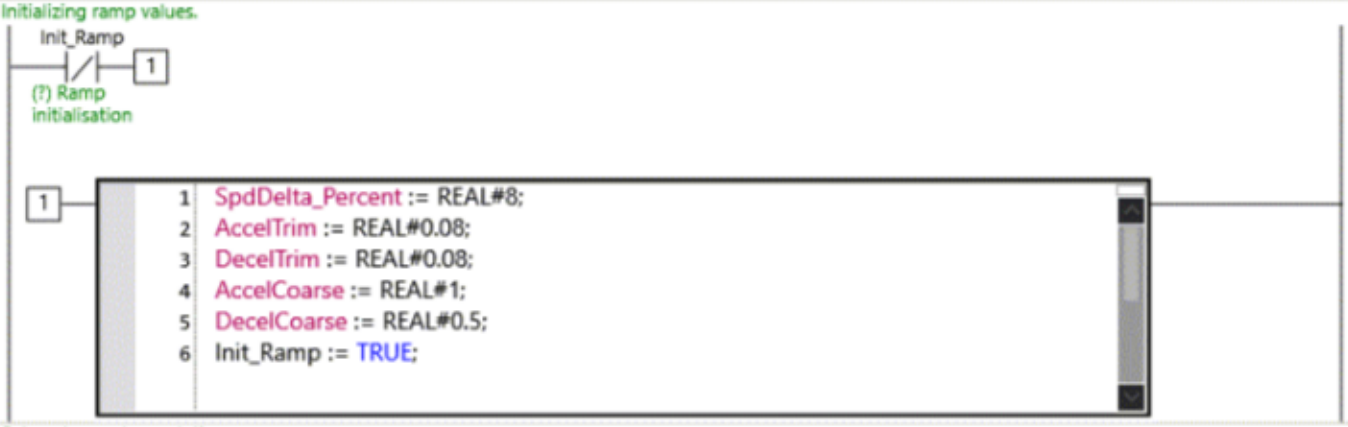
1-7-1-1-2.General

0 Remote 10.146.233.11 ... 1s clock.

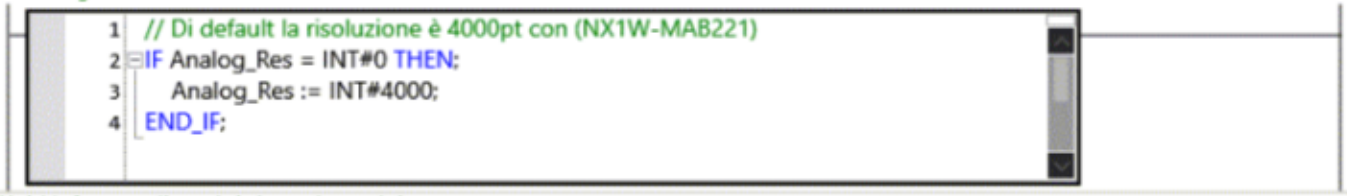
1 100ms clock.

2 10ms clock.

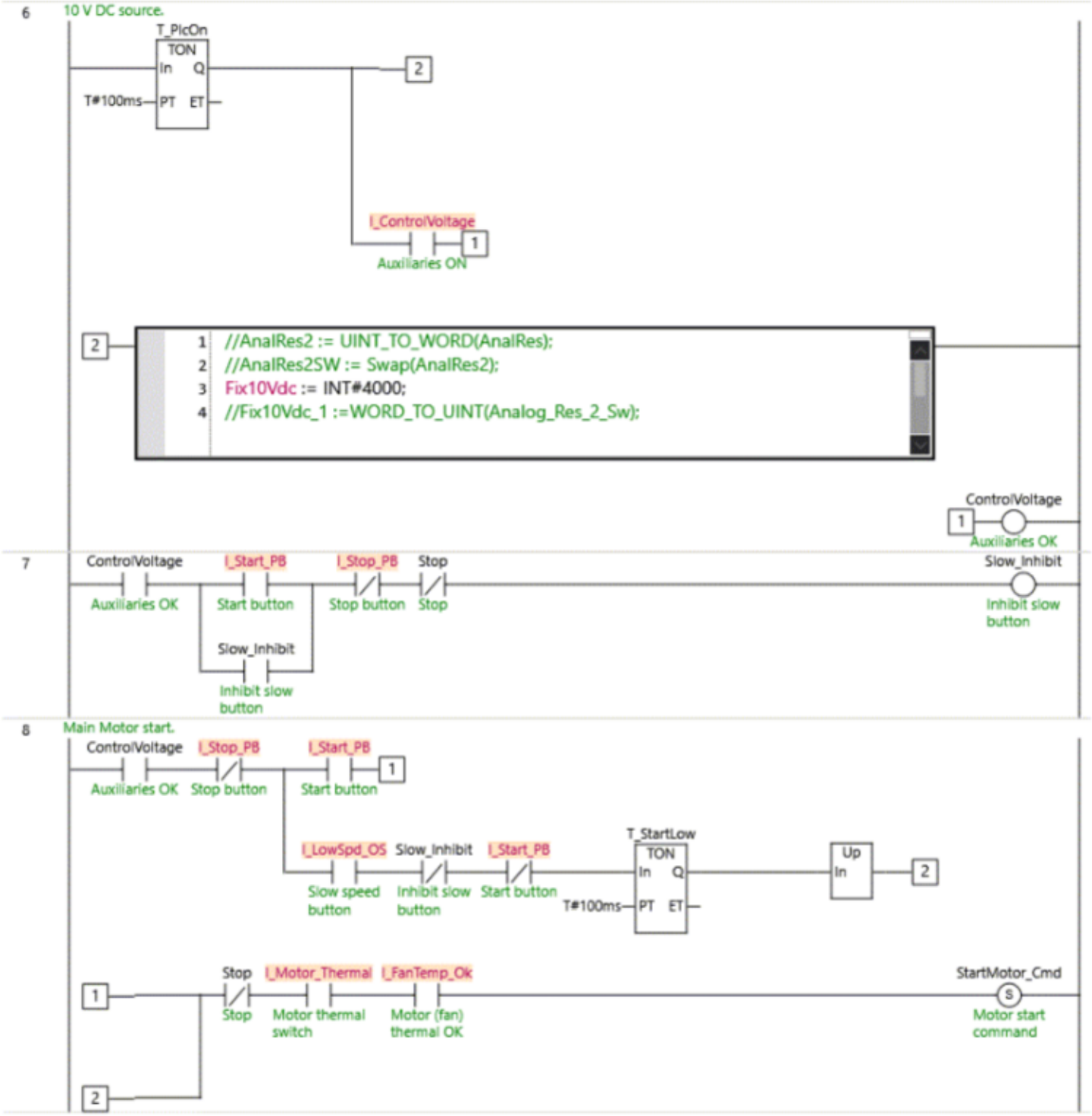
3 Initializing ramp values.

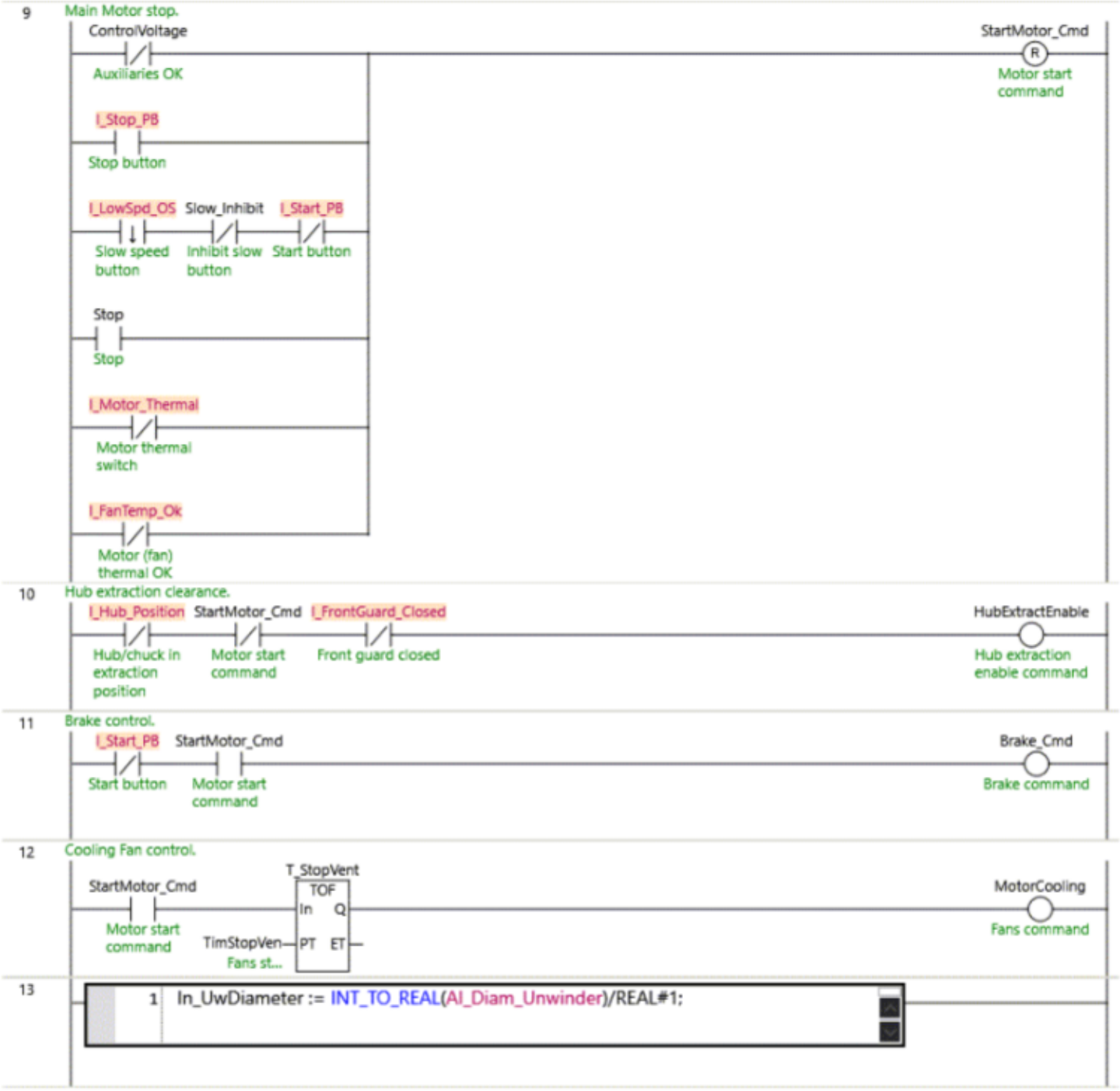


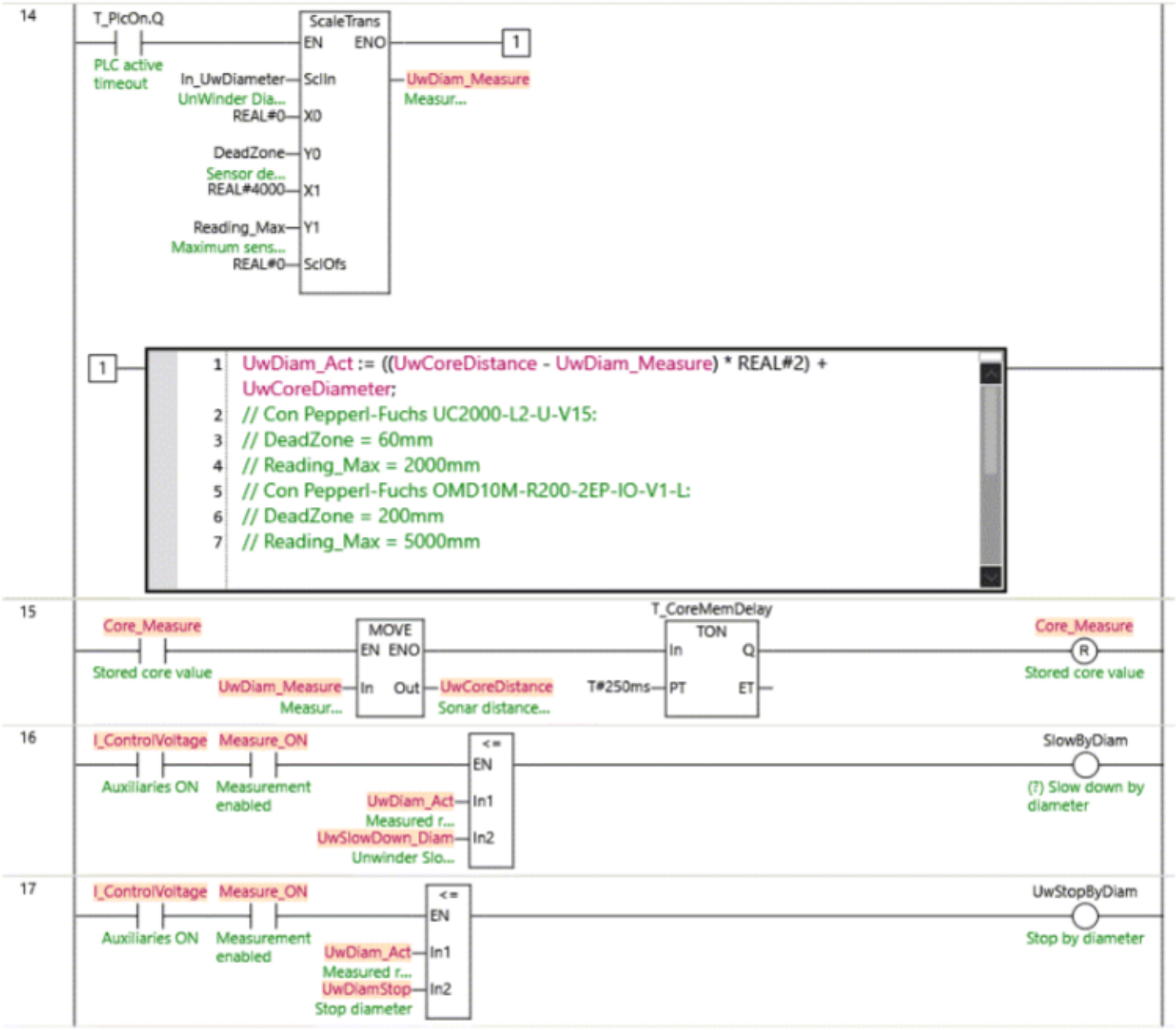
4 Set analog scale resolution.



5 Variable for converting m/min to mm/sec and vice versa.


6
10 V DC source.
T_PlcOn
TON
In Q
T#100ms PT ET
2
I_ControlVoltage
1
Auxiliaries ON
2
1 //AnalRes2 := UINT_TO_WORD(AnalRes);
2 //AnalRes2SW := Swap(AnalRes2);
3 Fix10Vdc := INT#4000;
4 //Fix10Vdc_1 :=WORD_TO_UINT(Analog_Res_2_Sw);
ControlVoltage
1
Auxiliaries OK
7
ControlVoltage
Auxiliaries OK
I_Start_PB
Start button
I_Stop_PB
Stop button
Stop
Stop
Slow_Inhibit
Inhibit slow button
Slow_Inhibit
Inhibit slow button
8
Main Motor start.
ControlVoltage
Auxiliaries OK
I_Stop_PB
Stop button
I_Start_PB
Start button
1
I_LowSpd_OS
Slow speed button
Slow_Inhibit
Inhibit slow button
I_Start_PB
Start button
T_StartLow
TON
In Q
T#100ms PT ET
Up
In
2
1
Stop
Stop
I_Motor_Thermal
Motor thermal switch
I_FanTemp_Ok
Motor (fan) thermal OK
StartMotor_Cmd
S
Motor start command
2


9
Main Motor stop.
ControlVoltage
Auxiliaries OK
I_Stop_PB
Stop button
I_LowSpd_OS
Slow speed button
Slow_Inhibit
Inhibit slow button
I_Start_PB
Start button
Stop
Stop
I_Motor_Thermal
Motor thermal switch
I_FanTemp_Ok
Motor (fan) thermal OK
StartMotor_Cmd
R
Motor start command
10
Hub extraction clearance.
I_Hub_Position
Hub/chuck in extraction position
StartMotor_Cmd
Motor start command
I_FrontGuard_Closed
Front guard closed
HubExtractEnable
Hub extraction enable command
11
Brake control.
I_Start_PB
Start button
StartMotor_Cmd
Motor start command
Brake_Cmd
Brake command
12
Cooling Fan control.
StartMotor_Cmd
Motor start command
T_StopVent
TOF
In
Q
TimStopVen
Fans st...
PT
ET
MotorCooling
Fans command
13
1
In_UwDiameter := INT_TO_REAL(AI_Diam_Unwinder)/REAL#1;


14
T_PlcOn.Q
PLC active timeout
ScaleTrans
EN
ENO
1
In_UwDiameter
UnWinder Dia...
REAL#0
SclIn
X0
DeadZone
Y0
Sensor de...
REAL#4000
X1
Reading_Max
Maximum sens...
REAL#0
Y1
SclOfs
UwDiam_Measure
Measur...
1
1 UwDiam_Act := ((UwCoreDistance - UwDiam_Measure) * REAL#2) + UwCoreDiameter;
2 // Con Pepperl-Fuchs UC2000-L2-U-V15:
3 // DeadZone = 60mm
4 // Reading_Max = 2000mm
5 // Con Pepperl-Fuchs OMD10M-R200-2EP-IO-V1-L:
6 // DeadZone = 200mm
7 // Reading_Max = 5000mm
15
Core_Measure
Stored core value
MOVE
EN ENO
In Out
UwDiam_Measure
Measur...
UwCoreDistance
Sonar distance...
T_CoreMemDelay
TON
In Q
T#250ms
PT ET
Core_Measure
R
Stored core value
16
I_ControlVoltage
Auxiliaries ON
Measure_ON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwSlowDown_Diam
Unwinder Slo...
In2
SlowByDiam
(?) Slow down by diameter
17
I_ControlVoltage
Auxiliaries ON
Measure_ON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwDiamStop
Stop diameter
In2
UwStopByDiam
Stop by diameter

1-7-1-1-3.Motor_Manager

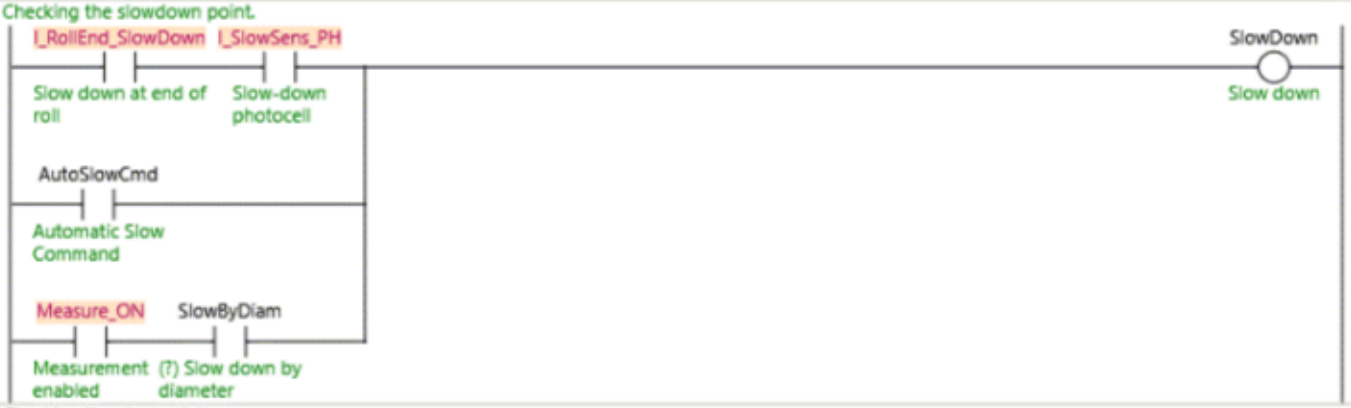
0 Enable encoder reading.

1 Calculation of mm per encoder pulse.

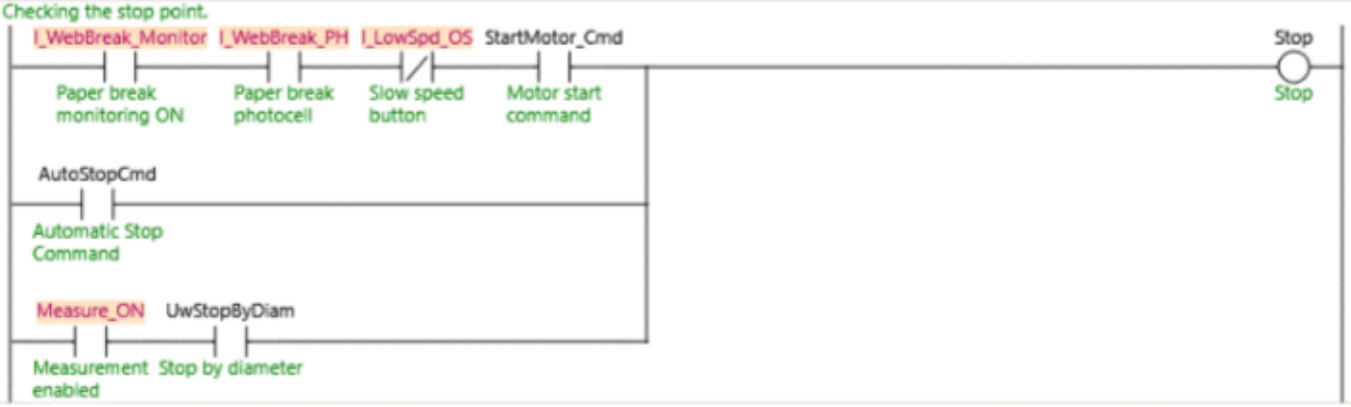
```
IF Encoder_Pulse = INT#0 THEN;
    Encoder_Pulse :=INT#1;
END_IF;
//Diametro standard ruota 400mm
IF  EncoderWheel_Diam = REAL#0 THEN;
    EncoderWheel_Diam := REAL#400;
END_IF;

EncoderPulse_to_mm := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) *
REAL#4);
```

2 Reset encoder counter.

3 Checking the slowdown point.



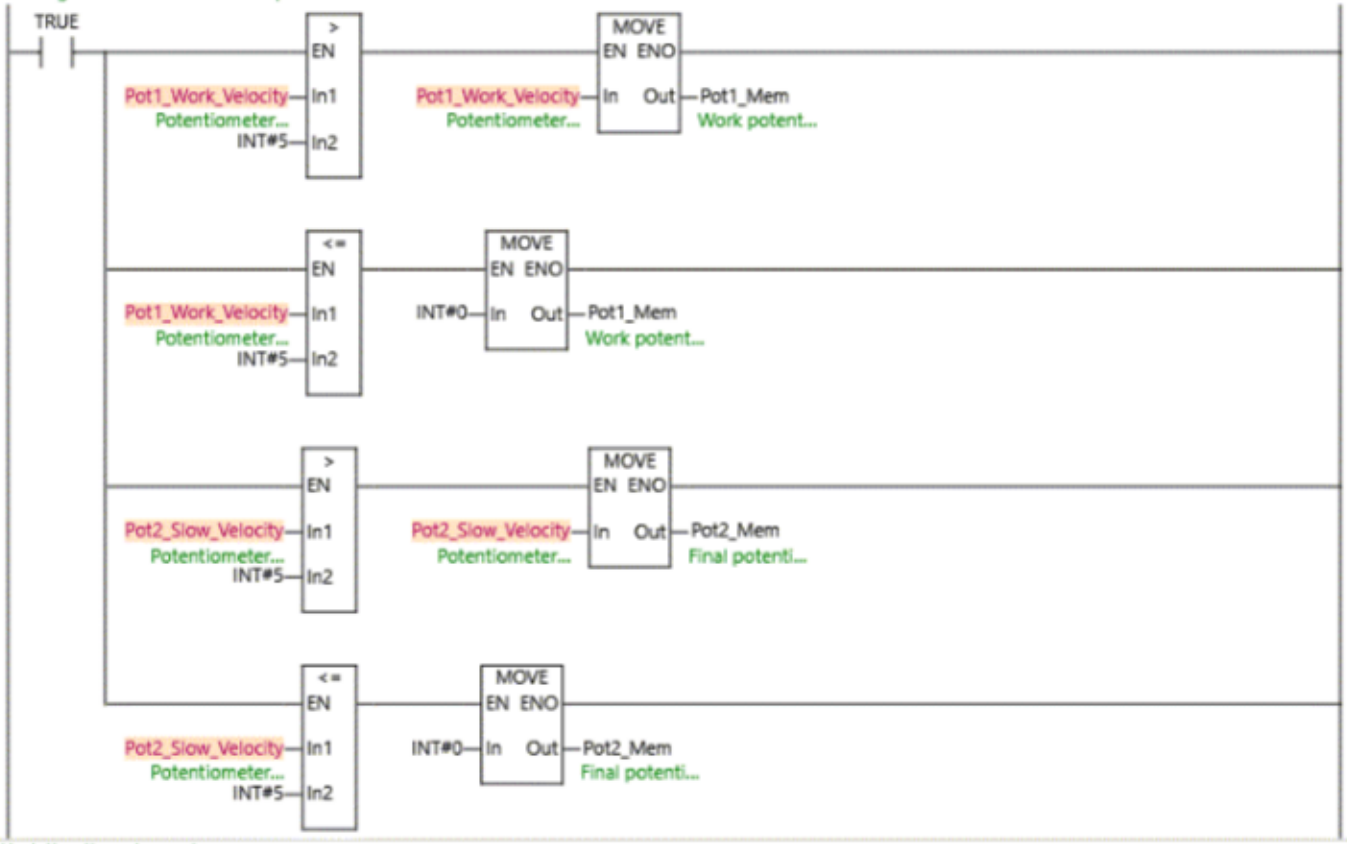
4 Checking the stop point.



5 Check the maximum and minimum values for working speed and slow speed.

```
IF VelMax < REAL#600 THEN;
    VelMax := REAL#600;
ELSIF VelMax > REAL#1000 THEN;
    VelMax := REAL#1000;
END_IF;
Vel_Max_mms := VelMax * REAL#1000 / REAL#60;

IF JogSpd_SP < REAL#10 THEN;
    JogSpd_SP := REAL#10;
ELSIF JogSpd_SP > REAL#70 THEN;
    JogSpd_SP := REAL#70;
END_IF;

Output_Ratio := (VelMax * Mt_To_mm) / UINT_TO_REAL(Analog_Res): //REAL#4000;
Input_Ratio := UINT_TO_REAL(Analog_Res) / VelMax ;
```

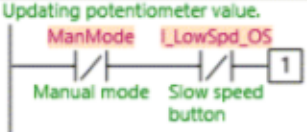
6 Reading the current value of the potentiometers.



7 Updating the set speed.

```
IF ManMode = FALSE THEN;

    Speed_Display := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
    LowSpd_Display := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
ELSE;
    Speed_Display := Pot1_Mem / (Analog_Res  /UINT#40); // INT#40;
    LowSpd_Display := Pot2_Mem / (Analog_Res / UINT#40); // INT#40;

END_IF;
```

8 Updating potentiometer value.




```
IF SlowDown = FALSE THEN;
    VelSet := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
ELSE;
    VelSet := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
END_IF;
VelSet_mms := VelSet * Mt_To_mm;
```

9 Slow speed.

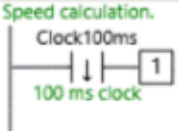
```
VelSet := JogSpd_SP;
VelSet_mms := VelSet * Mt_To_mm;
```

10 Check the difference between the set speed and the detected speed.

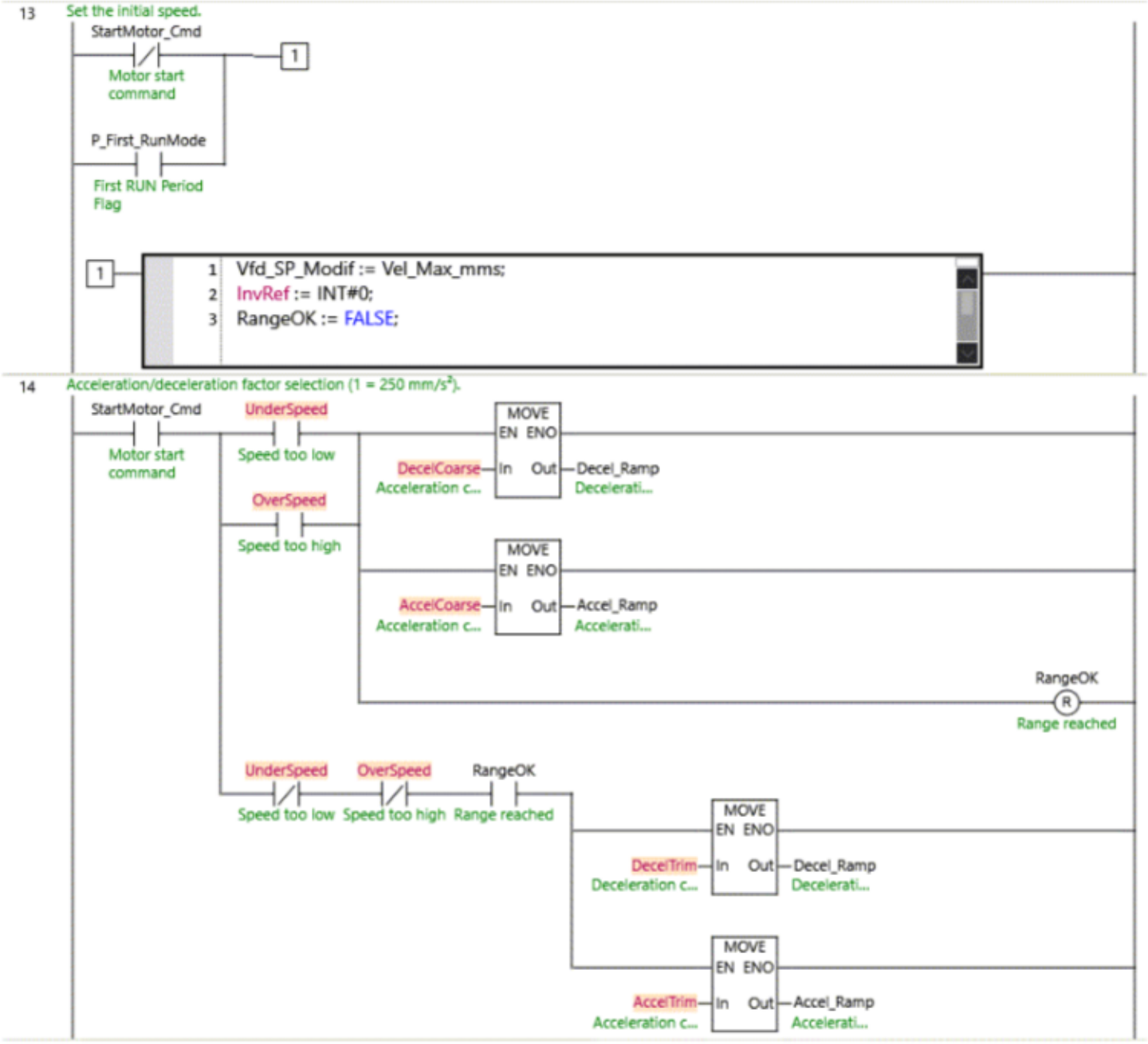
```
//SpdDelta := VelSet_mms * SpdDelta_Percent / REAL#100;
SpdDelta := SpdDelta_Percent;
SpdDeltaMax := VelSet + SpdDelta;
SpdDeltaMin := VelSet - SpdDelta;
// Troppo veloce.
IF Velocity_M_min > SpdDeltaMax THEN;
    OverSpeed := TRUE;
ELSE
    OverSpeed := FALSE;
END_IF;
// Troppo lento.
IF Velocity_M_min < SpdDeltaMin THEN;
    UnderSpeed := TRUE;
ELSE
    UnderSpeed := FALSE;
END_IF;
```

11 Read the current value of the encoder.

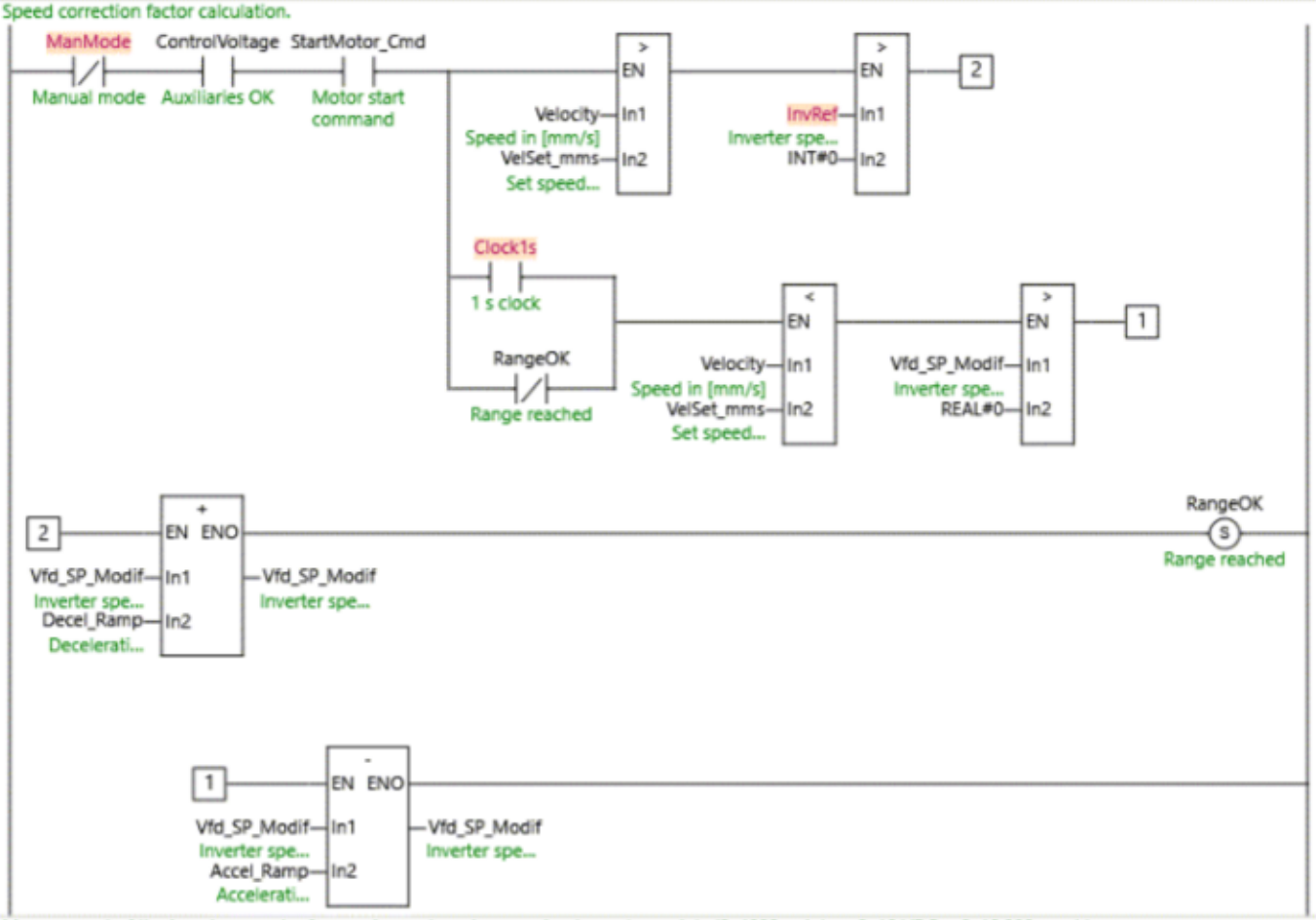
12 Speed calculation.



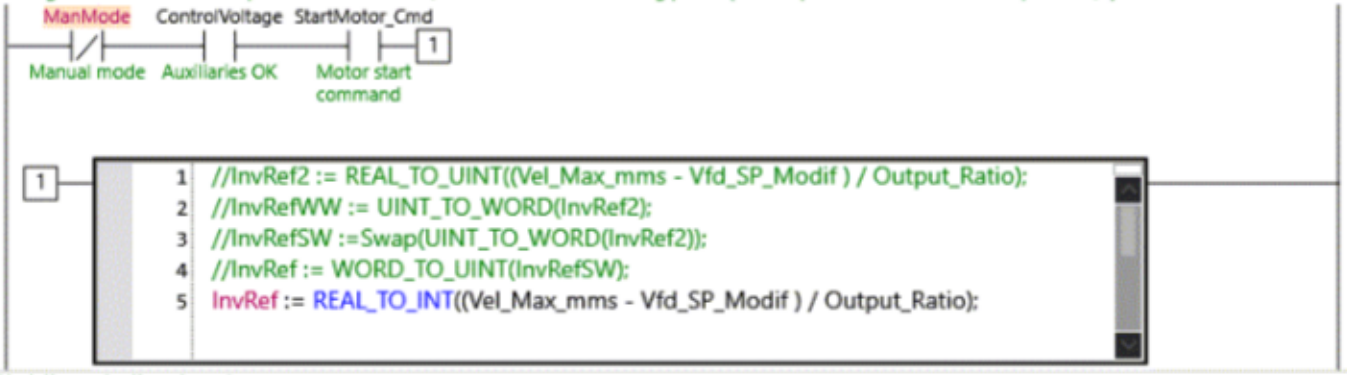

```
DeltaPulse := Encoder_Pos - MemPos;
Velocity := (DINT_TO_REAL(DeltaPulse) * EncoderPulse_to_mm) * REAL#20; //
Campionamento ogni 1/20sec. (50ms)
Velocity_M_min := Velocity / Mt_To_mm;
```


13
Set the initial speed.
StartMotor_Cmd
Motor start command
1
P_First_RunMode
First RUN Period Flag
1
1 Vfd_SP_Modif := Vel_Max_mms;
2 InvRef := INT#0;
3 RangeOK := FALSE;
14
Acceleration/deceleration factor selection (1 = 250 mm/s²).
StartMotor_Cmd
Motor start command
UnderSpeed
Speed too low
OverSpeed
Speed too high
MOVE
EN ENO
DecelCoarse
Acceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelCoarse
Acceleration c...
In Out
Accel_Ramp
Accelerati...
RangeOK
R
Range reached
UnderSpeed
Speed too low
OverSpeed
Speed too high
RangeOK
Range reached
MOVE
EN ENO
DecelTrim
Deceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelTrim
Acceleration c...
In Out
Accel_Ramp
Accelerati...

15 Speed correction factor calculation.

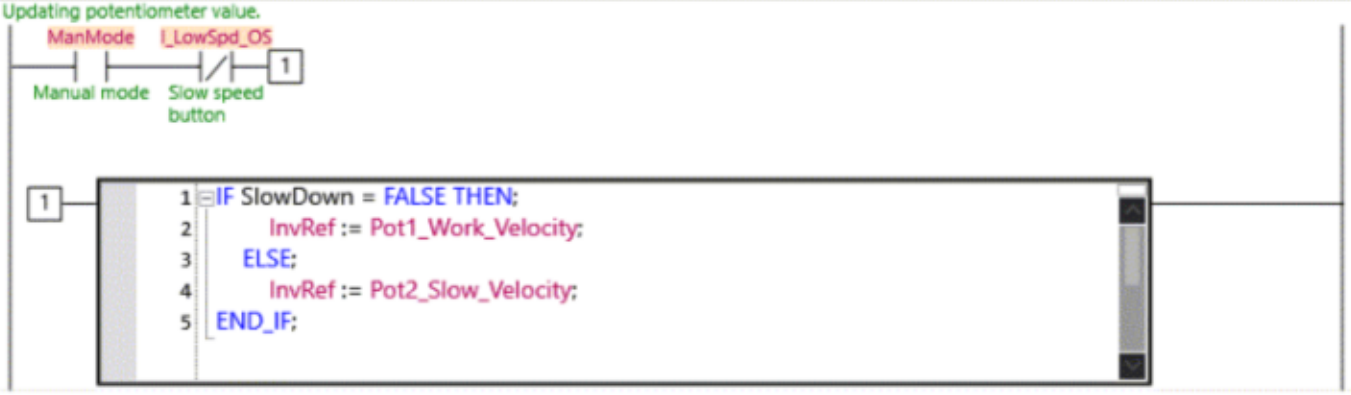


16 Management of the inverter speed reference in mm/s and conversion to analog points (0–4000 points = 0–10 VDC = 0–10,000 mm/s).

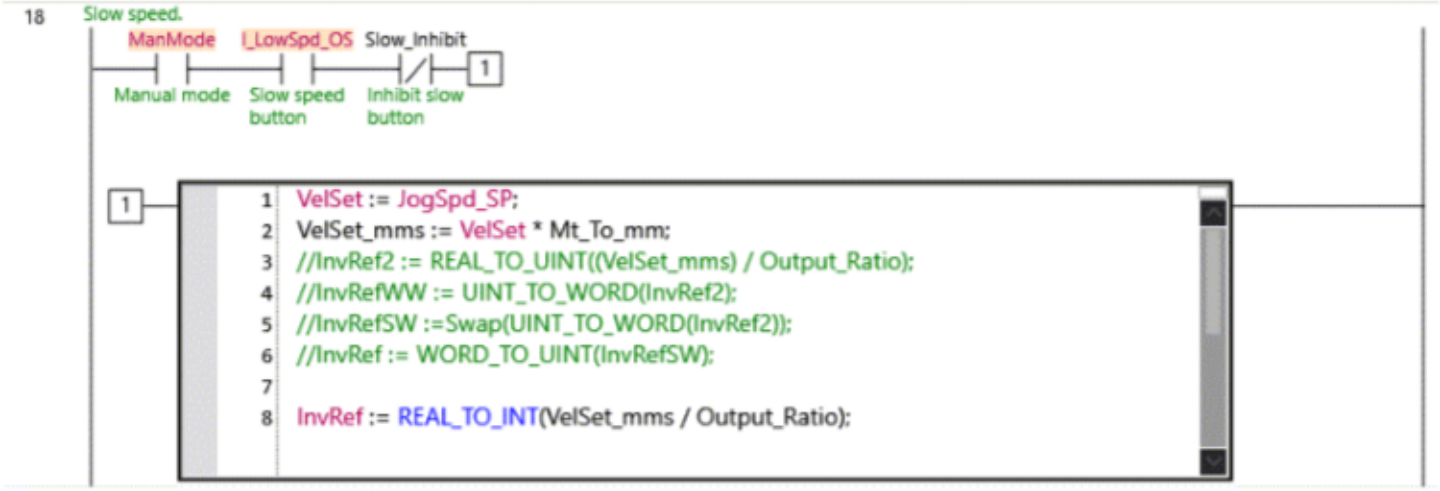



```
//InvRef2 := REAL_TO_UINT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
//InvRefWW := UINT_TO_WORD(InvRef2);
//InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
//InvRef := WORD_TO_UINT(InvRefSW);
InvRef := REAL_TO_INT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
```

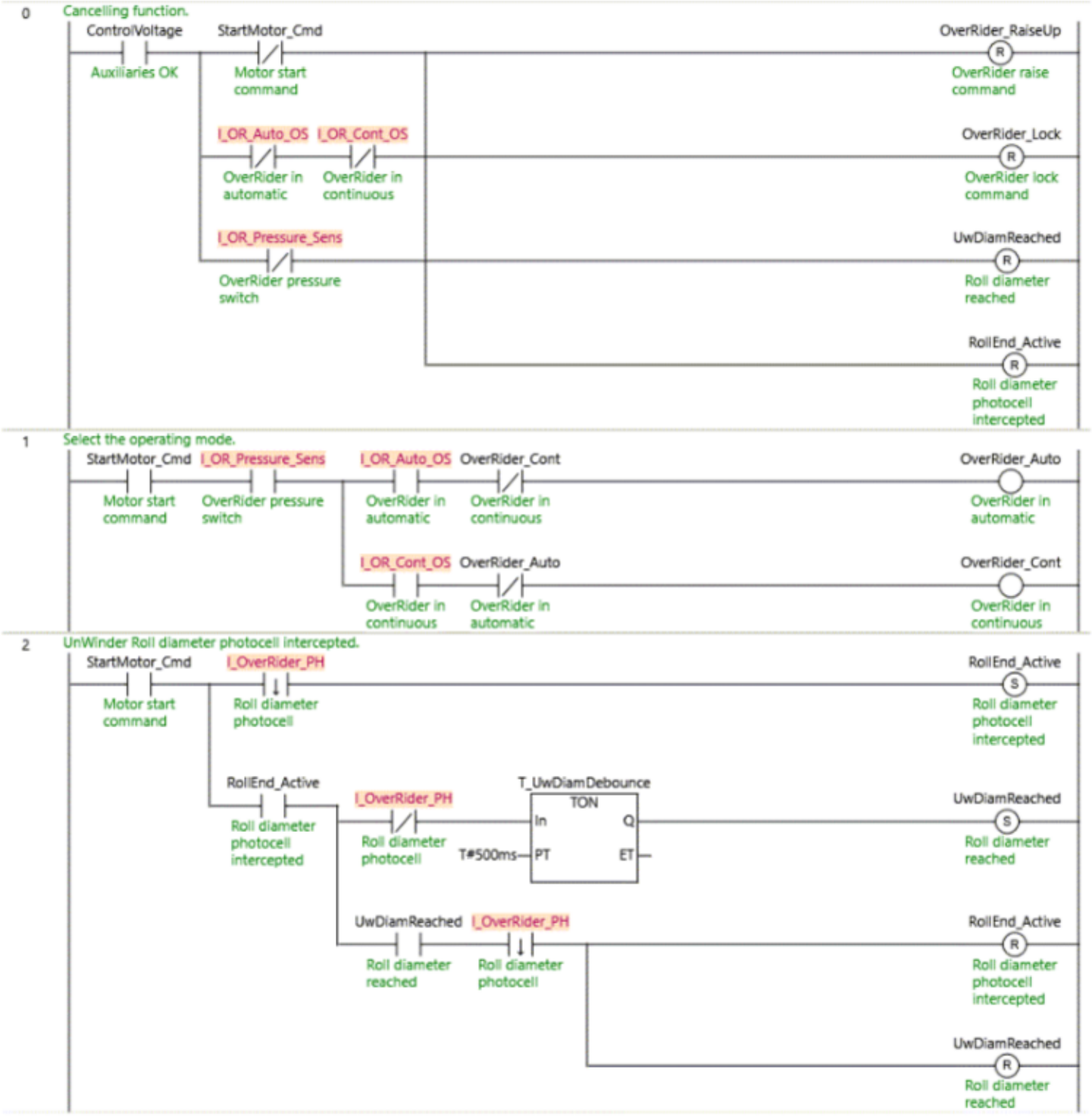
17 Updating potentiometer value.



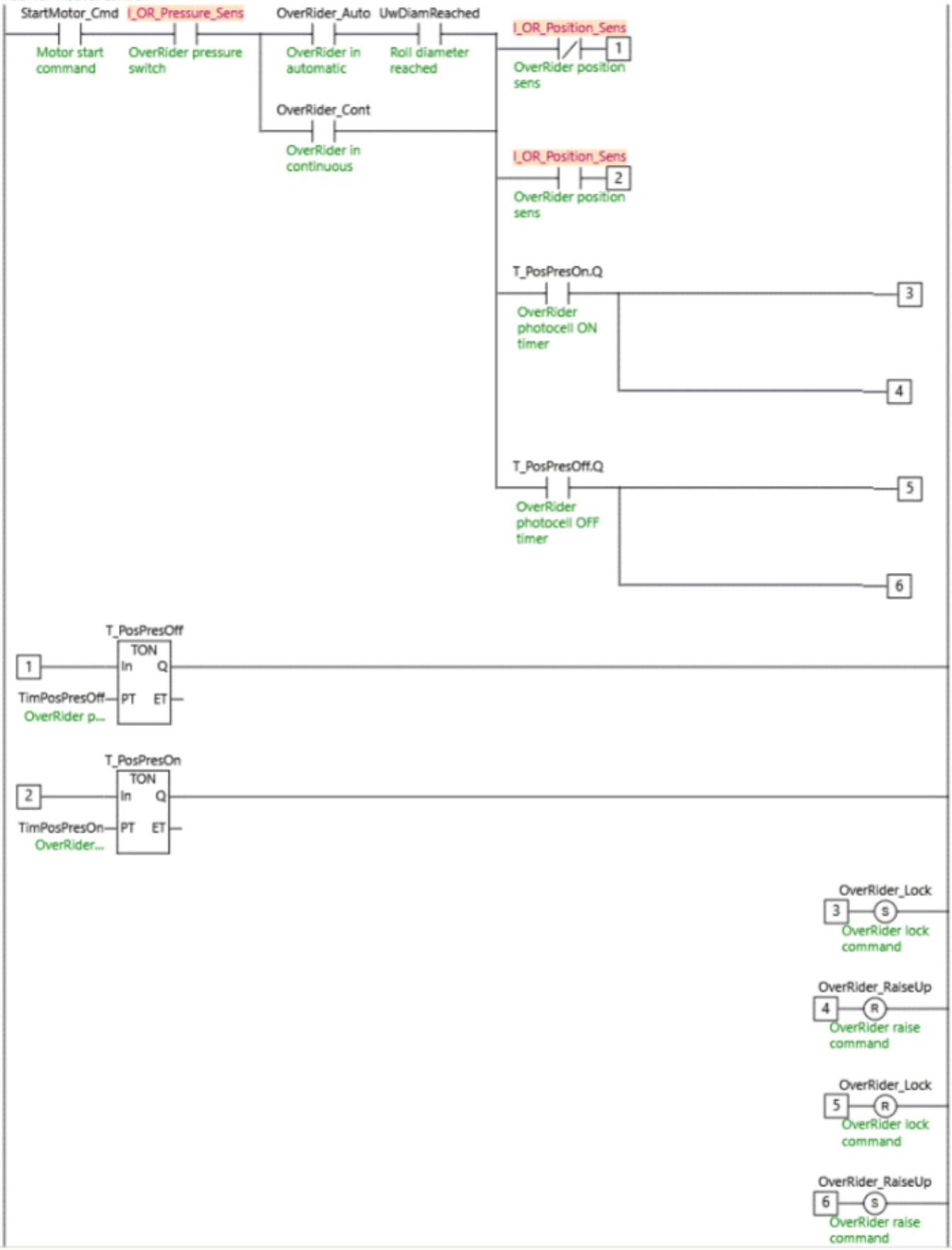

```
IF SlowDown = FALSE THEN;
    InvRef := Pot1_Work_Velocity;
ELSE;
    InvRef := Pot2_Slow_Velocity;
END_IF;
```


18
Slow speed.
ManMode
I_LowSpd_OS
Slow_Inhibit
1
Manual mode
Slow speed button
Inhibit slow button
1
1 VelSet := JogSpd_SP;
2 VelSet_mms := VelSet * Mt_To_mm;
3 //InvRef2 := REAL_TO_UINT((VelSet_mms) / Output_Ratio);
4 //InvRefWW := UINT_TO_WORD(InvRef2);
5 //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
6 //InvRef := WORD_TO_UINT(InvRefSW);
7
8 InvRef := REAL_TO_INT(VelSet_mms / Output_Ratio);

1-7-1-1-4.OverRider_Manager

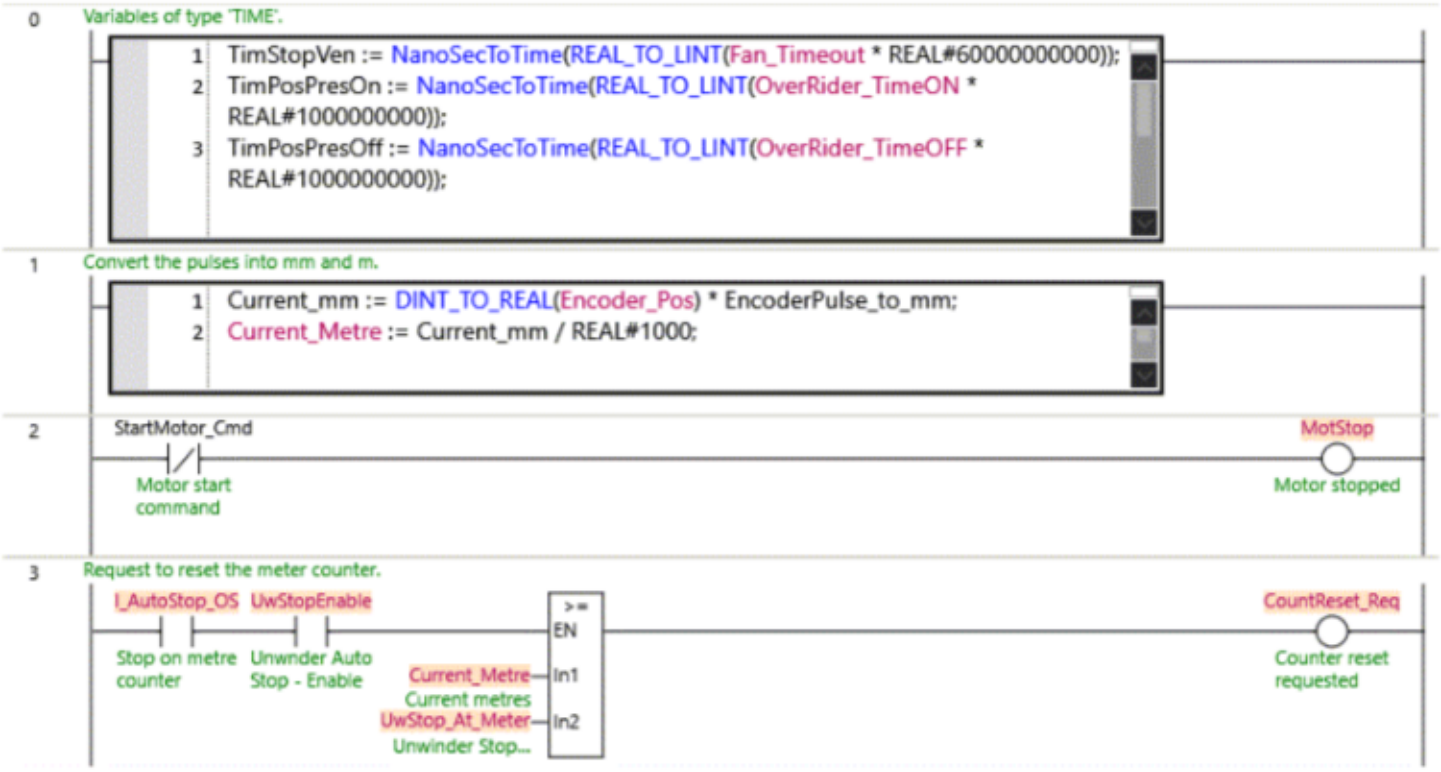


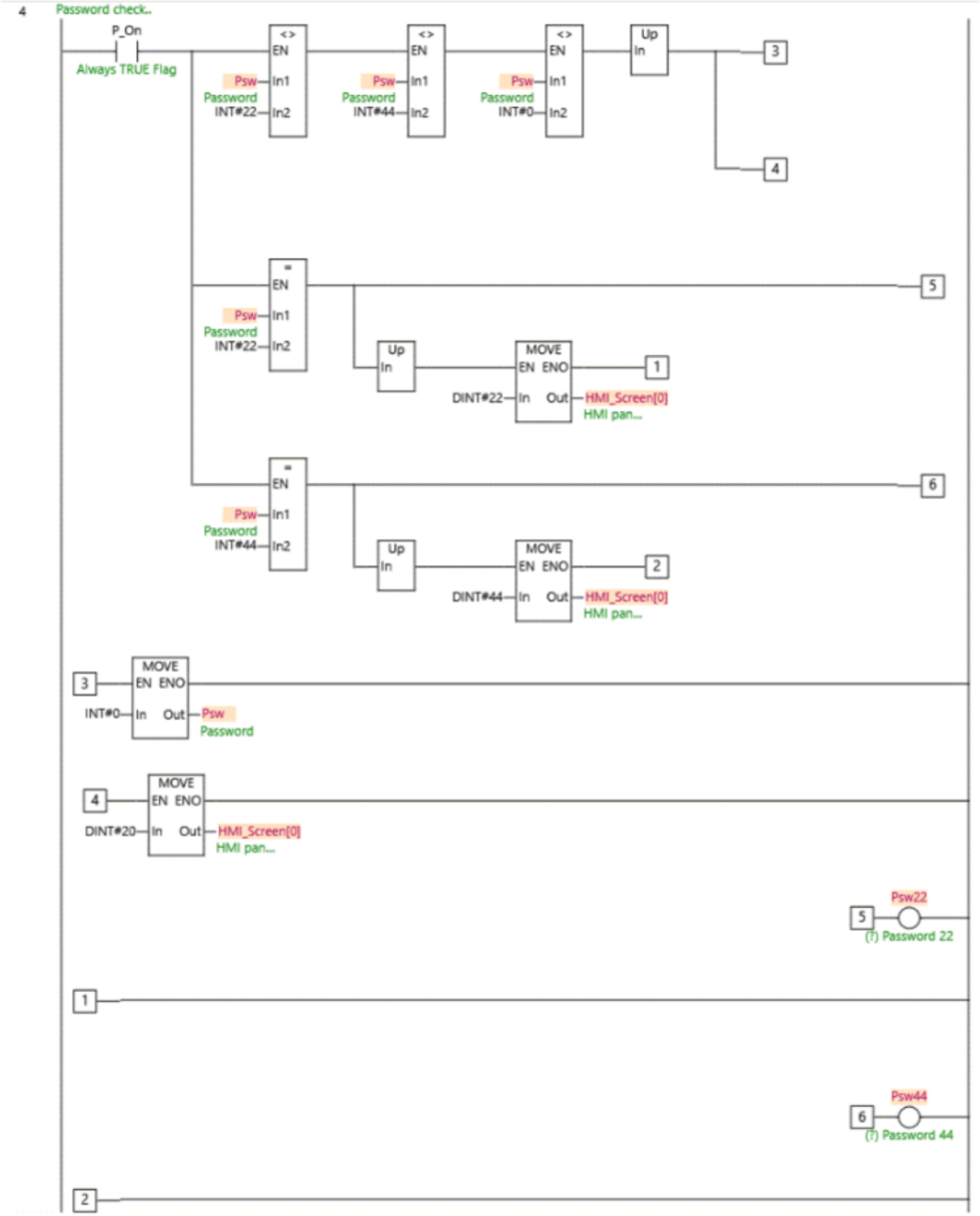
3 Rider roll Hoover control.

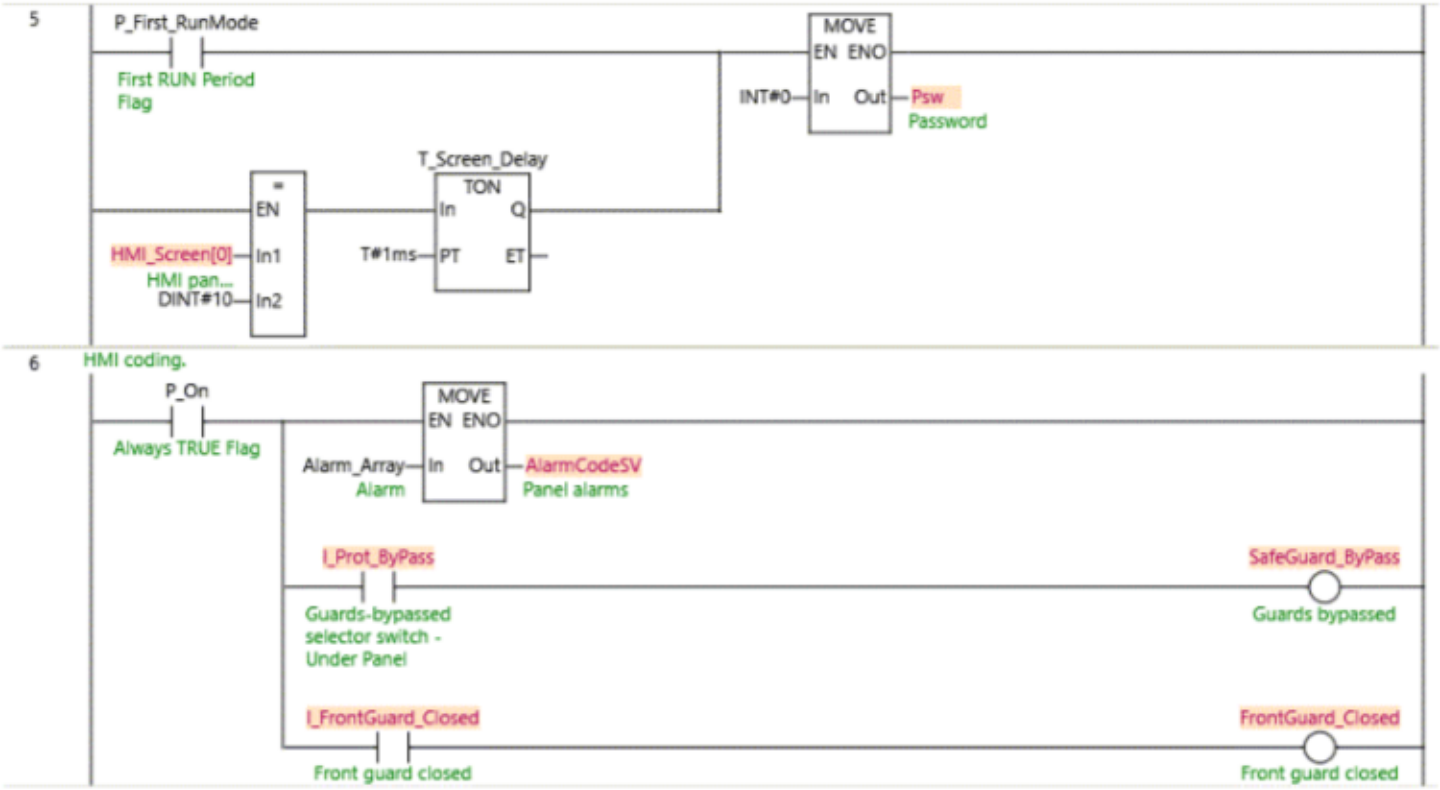


4
Rider roll raise command.
ControlVoltage
Auxiliaries OK
OverRider_RaiseUp
S
OverRider raise command
5
I_ControlVoltage
Auxiliaries ON
OverRider_RaiseUp
R
OverRider raise command

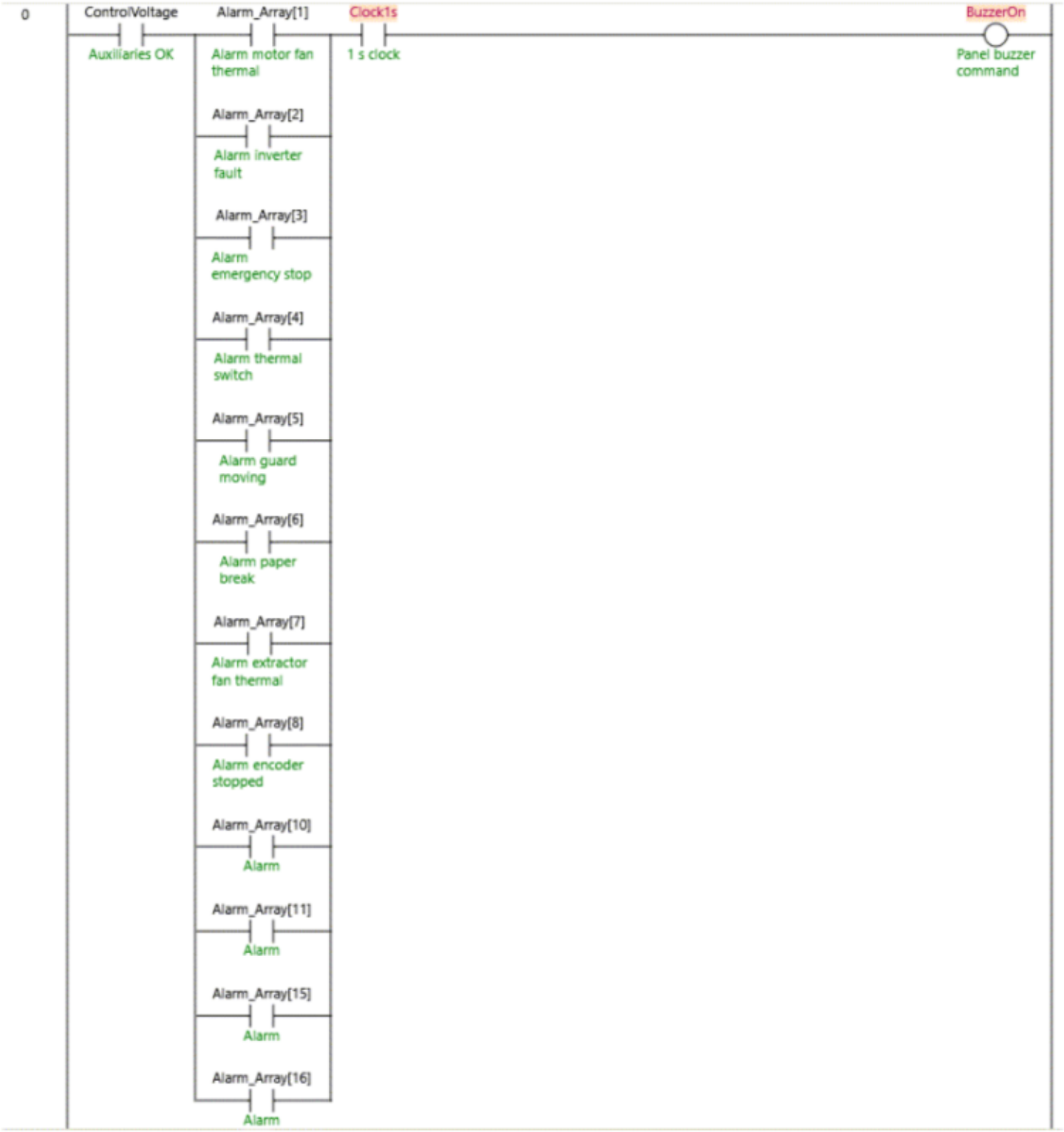
1-7-1-1-5.Console



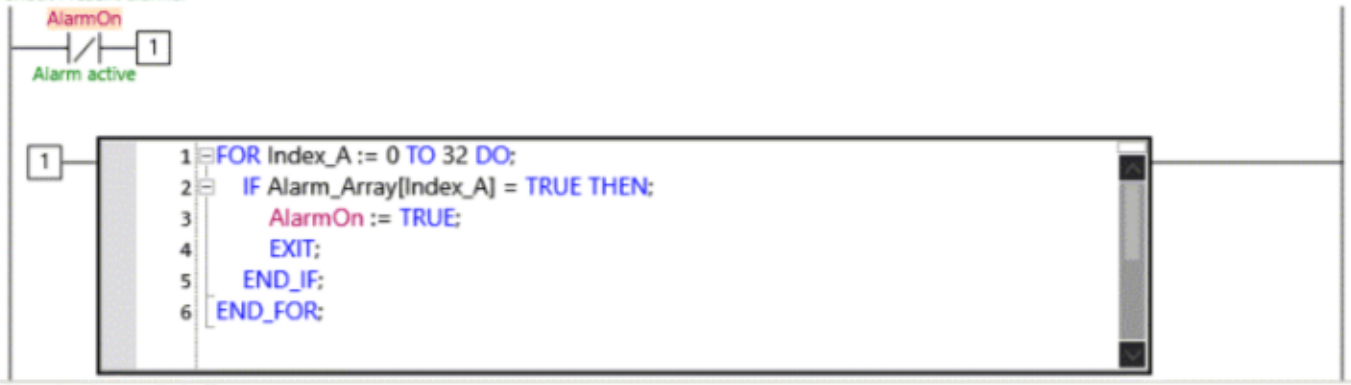

4
Password check..
P_On
Always TRUE Flag
<>
EN
In1
In2
Psw
Password
INT#22
INT#44
INT#0
Up
In
3
4
=
5
6
MOVE
EN ENO
In Out
1
2
DINT#22
DINT#44
HMI_Screen[0]
HMI pan...
INT#0
DINT#20
Psw
Psw22
(?) Password 22
Psw44
(?) Password 44


5
P_First_RunMode
First RUN Period Flag
MOVE
EN ENO
INT#0
In
Out
Psw
Password
T_Screen_Delay
=
EN
TON
In
Q
HMI_Screen[0]
HMI pan...
In1
T#1ms
PT
ET
DINT#10
In2
6
HMI coding.
P_On
Always TRUE Flag
MOVE
EN ENO
Alarm_Array
Alarm
In
Out
AlarmCodeSV
Panel alarms
I_Prot_ByPass
Guards-bypassed selector switch - Under Panel
SafeGuard_ByPass
Guards bypassed
I_FrontGuard_Closed
Front guard closed
FrontGuard_Closed
Front guard closed

1-7-1-1-6.Alarms



1 Check Present alarms.



2 Alarm has been called off.

3 Motor cooling Fan Alarm.

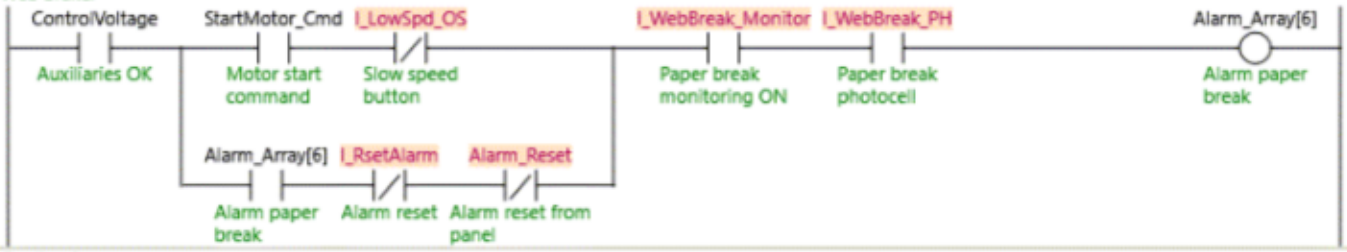
4 Inverter fault.

5 Machine in emergency mode.

6 Motor thermal cutout.

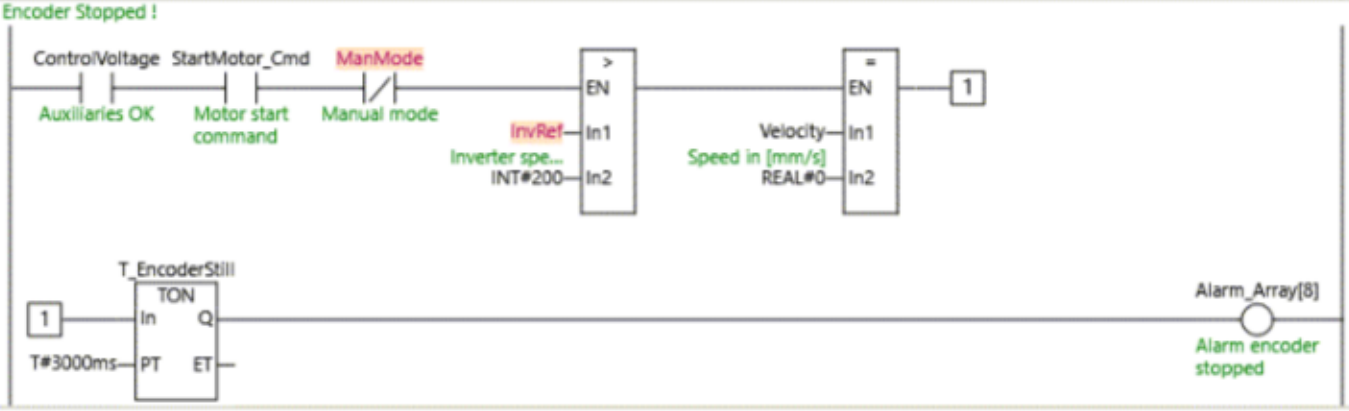
7

8 Web Brake.



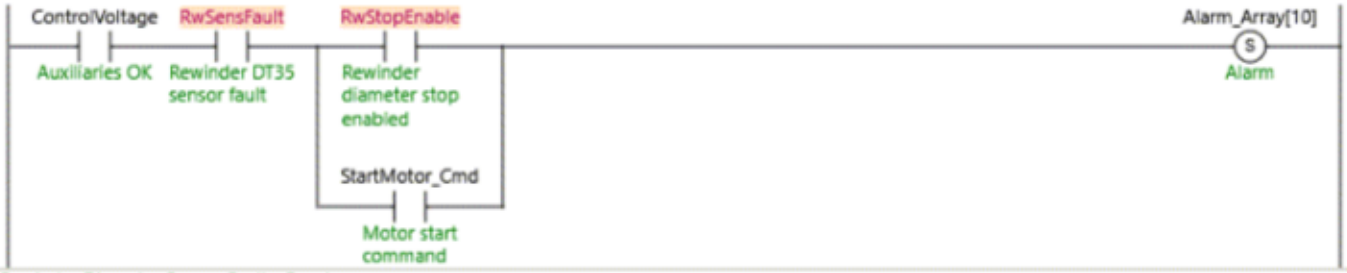
9 Cooling fan.

10 Encoder Stopped !



11 Open-ended roll holder joint.

12 Rewinder Diameter Sensor Fault.



13 Rewinder Diameter Sensor Fault - Reset

14 Rewinder max diameter Machine protection.

15 Rewinder max diameter Machine protection - Reset

16 Rewinder learning fault

17 Rewinder learning fault - Reset

18 Hub not in position.

19 Micro a fune attivo.

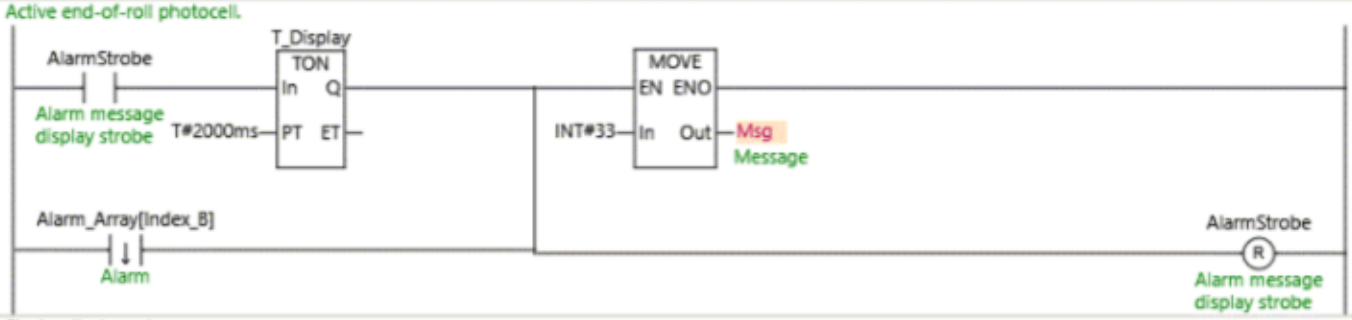
20 Slow-down photocell active.

21 End-of-roll Alarm.

22 Unwinder Roll is Empty

23 Unwinder Roll is Empty - Reset

24 Active end-of-roll photocell.

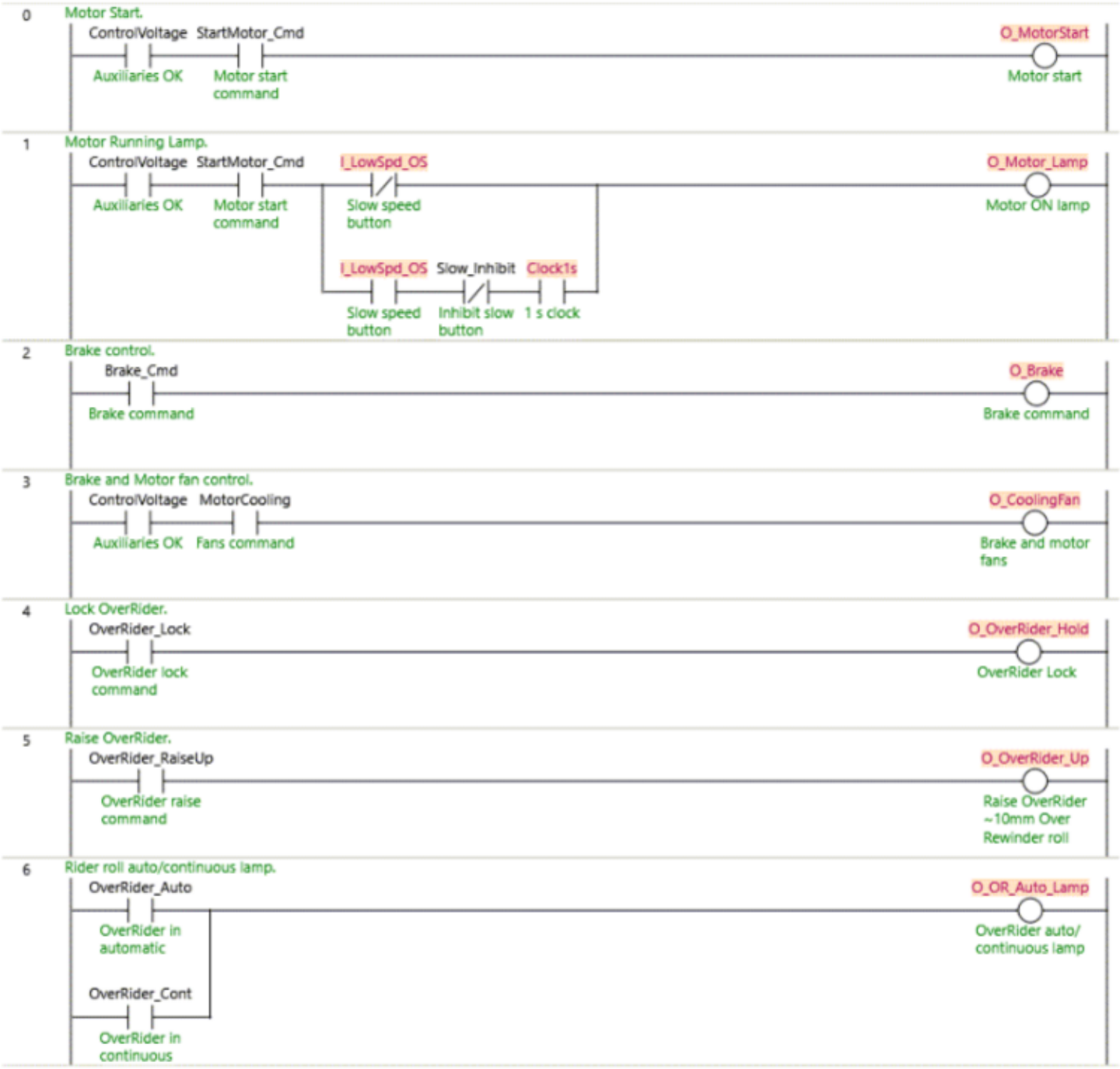


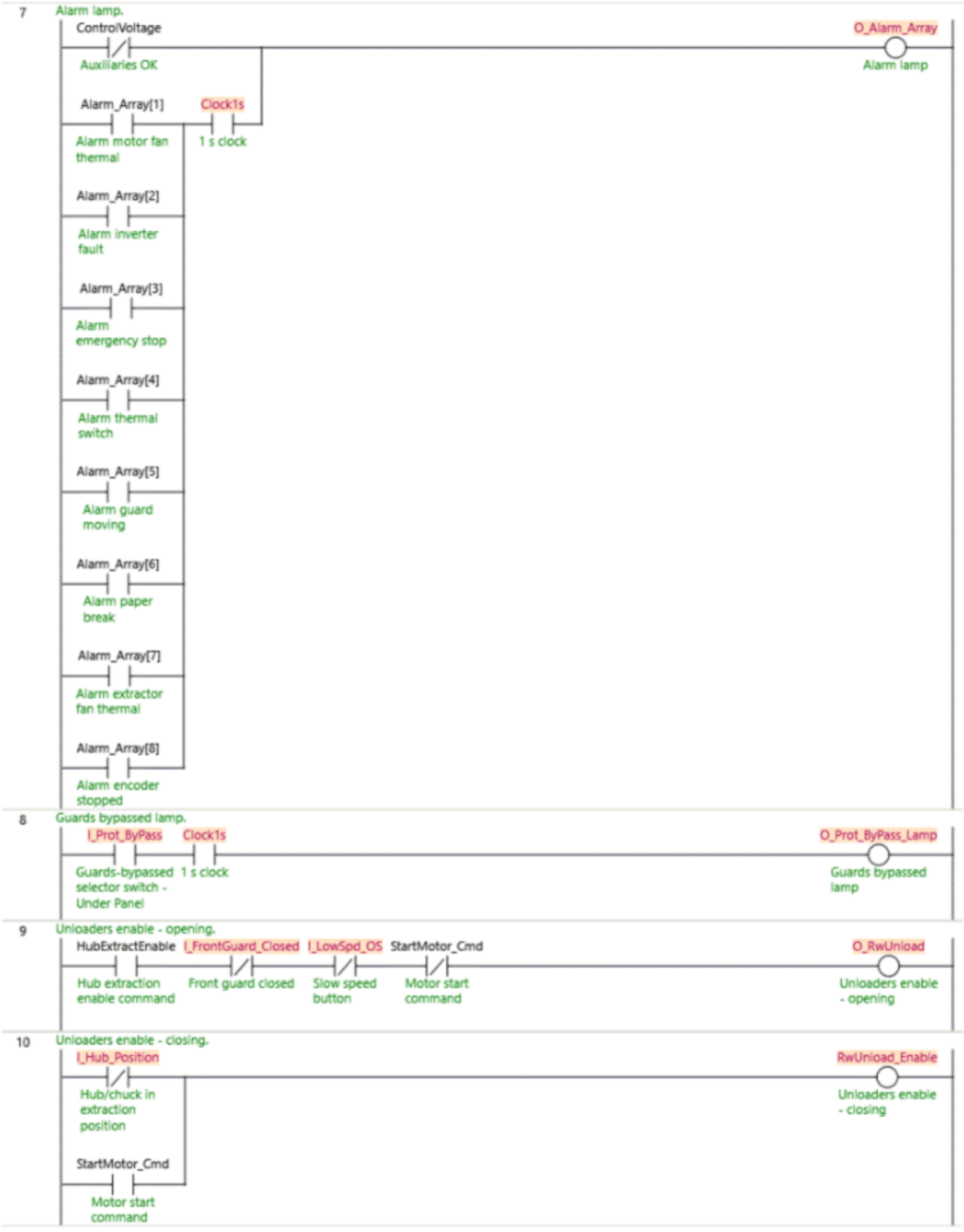
25 Strobe displays alarm message.

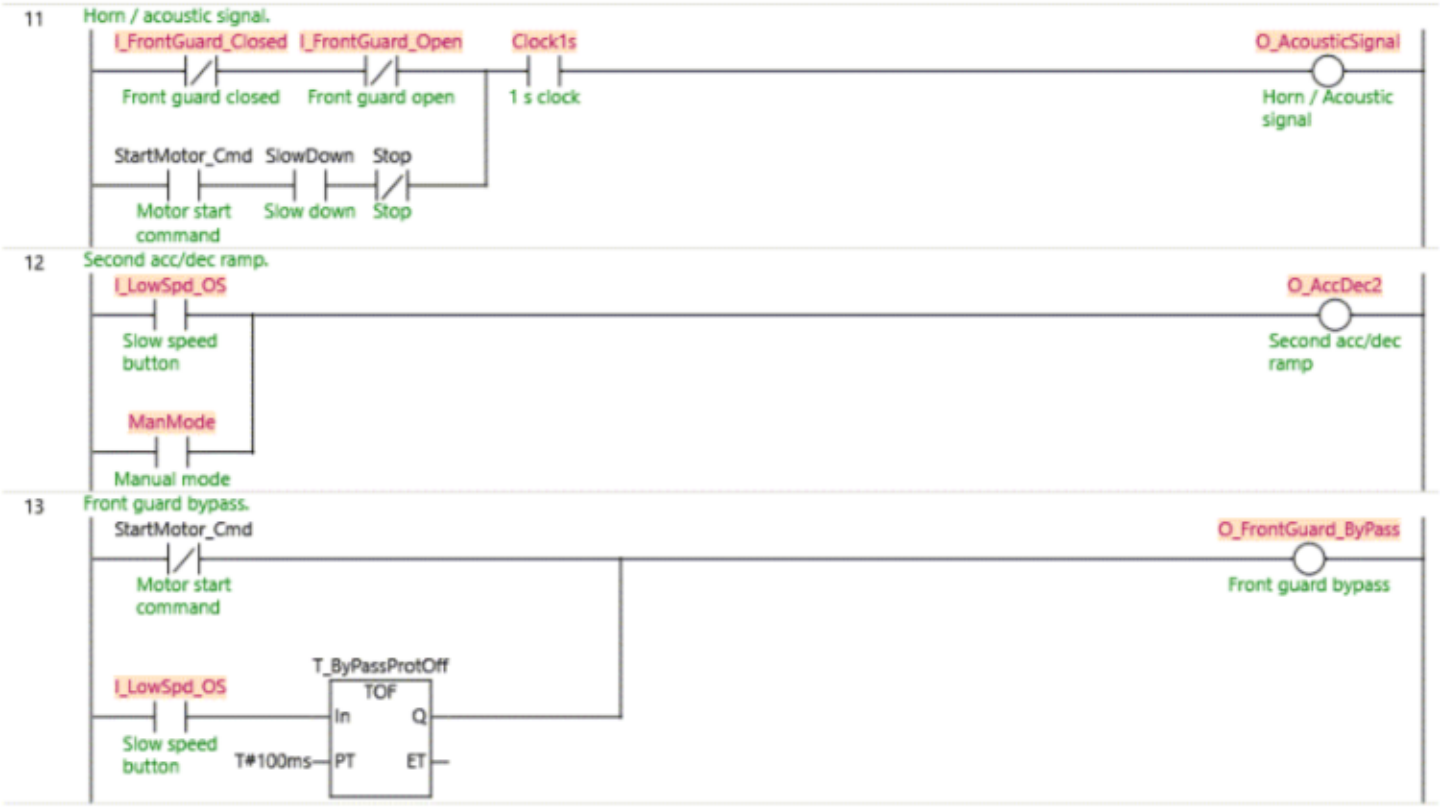
26 Alarm presence check cycle.

27 Initializing alarm message.

1-7-1-1-7.Output_Manager




7
Alarm lamp.
ControlVoltage
Auxiliaries OK
O_Alarm_Array
Alarm lamp
Alarm_Array[1]
Alarm motor fan thermal
Clock1s
1 s clock
Alarm_Array[2]
Alarm inverter fault
Alarm_Array[3]
Alarm emergency stop
Alarm_Array[4]
Alarm thermal switch
Alarm_Array[5]
Alarm guard moving
Alarm_Array[6]
Alarm paper break
Alarm_Array[7]
Alarm extractor fan thermal
Alarm_Array[8]
Alarm encoder stopped
8
Guards bypassed lamp.
I_Prot_ByPass
Guards-bypassed selector switch - Under Panel
Clock1s
1 s clock
O_Prot_ByPass_Lamp
Guards bypassed lamp
9
Unloaders enable - opening.
HubExtractEnable
Hub extraction enable command
I_FrontGuard_Closed
Front guard closed
I_LowSpd_OS
Slow speed button
StartMotor_Cmd
Motor start command
O_RwUnload
Unloaders enable - opening
10
Unloaders enable - closing.
I_Hub_Position
Hub/chuck in extraction position
StartMotor_Cmd
Motor start command
RwUnload_Enable
Unloaders enable - closing


11
Horn / acoustic signal.
I_FrontGuard_Closed
Front guard closed
I_FrontGuard_Open
Front guard open
Clock1s
1 s clock
O_AcousticSignal
Horn / Acoustic signal
StartMotor_Cmd
Motor start command
SlowDown
Slow down
Stop
Stop
12
Second acc/dec ramp.
I_LowSpd_OS
Slow speed button
O_AccDec2
Second acc/dec ramp
ManMode
Manual mode
13
Front guard bypass.
StartMotor_Cmd
Motor start command
O_FrontGuard_ByPass
Front guard bypass
T_ByPassProtOff
TOF
I_LowSpd_OS
Slow speed button
In
Q
T#100ms
PT
ET

1-7-1-1-8.AutoStop_Control

0

```
(*
================================================================
===============
  (*
================================================================
====================
  AutoStop_Control  v6d  -  Automatic slow-down / stop
  Project : 04820KA001 Mono Canada II  (NX1P2, PrimaryTask 4 ms)
  Rewinder (Rw) : SICK DT35 0-10 V -> AI_Diam_Rewinder (AD2603 Ch2)
  Unwinder (Uw) : meter counter mode + sonar diameter (UwDiam_Act)

  NAMING  Uw_ = unwinder   Rw_ = rewinder
        _mm = millimeters   _m = meters
        _Pls = encoder pulses (counts)   _Pps = pulses per second
        _Pps2 = pulses per second squared

  v6d: exact 100 ms tick (25 x 4 ms scan counter, the self-resetting
       TON ran ~104-108 ms -> speed ~4-8 % too high)
      standstill = <= 2 counts / 100 ms (encoder jitter)
      stop learning skipped after jog / manual / slow-button runs and
       when the auxiliaries drop (E-stop): RunWasJog, StopLearnValid
      work / final speed from the potentiometers in BOTH modes
       (Speed_Display / LowSpd_Display are not m/min in ManMode)
      slow-down length includes the DecelTrim tail inside the
       SpdDelta_Percent band (Motor_Manager rungs 14/15)
  v6c: learning supervision -> RwLearnFault (alarm 16), learned paper
       thickness RwThick_mm, plausibility limits for K and stop decel
  v6b: RwMaxSlowAtDiam display + RwMaxSlowFallback (slow-down warning
       while RwMat_K is not learned yet)
  v6 : consistent internal names, units in names.
      Globals renamed: RwRemain_m, RwStopLen_m, RwSlowLen_m,
                  RwSlowMargin_m, UwSlowMargin_m
  v5 : max diameter protection (always active), sensor plausibility,
      12SA5 master (section 9), alarms in section Alarms (ladder)
  v4b: AUTO interlock + single HMI display (SlowAtDisp / SlowAtMode)

  ================================================================
  ==================== *)

(* ---- 1. 100 ms tick, mm per encoder pulse ------------------------------ *)
(* v6d: 25 scans x 4 ms PrimaryTask = exactly 100 ms.                    *)
(*      Change 25 if the PrimaryTask period is changed.                  *)
TickCnt := TickCnt + 1;
IF TickCnt >= 25 THEN
    TickCnt := 0;
    Tick := TRUE;
ELSE
    Tick := FALSE;
END_IF;

IF (Encoder_Pulse > 0) AND (EncoderWheel_Diam > 0.0) THEN
    MmPerPulse := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) * 4.0);
ELSE
    MmPerPulse := 0.0;
END_IF;

(* ---- 2. Rewinder DT35 -> filtered distance -> diameter ----------------- *)
```

```
52  RwDistRaw_mm := RwDist0V + (INT_TO_REAL(AI_Diam_Rewinder) / 4000.0)
53              * (RwDist10V - RwDist0V);
54
55  IF Tick THEN
56      IF NOT RwFiltInit THEN
57          RwDist := RwDistRaw_mm;
58          RwFiltInit := TRUE;
59      ELSE
60          RwDist := RwDist + RwFilter_K * (RwDistRaw_mm - RwDist);
61      END_IF;
62  END_IF;
63
64  RwSensFault := (AI_Diam_Rewinder < RwRawMin) OR (AI_Diam_Rewinder >
    RwRawMax)
65              OR (RwDist > RwAxisDist);
66
67  RwDiam := 2.0 * (RwAxisDist - RwDist);
68  IF RwDiam < 0.0 THEN
69      RwDiam := 0.0;
70  END_IF;
71  (* RwAxisDist = DT35 reference to rewind shaft centre (mm), set manually *)
72
73  (* ---- 3. Actual speed from encoder, 100 ms window ---------------------- *)
74  IF Tick THEN
75      IF NOT SpdInit THEN
76          EncPrev := Encoder_Pos;
77          SpdInit := TRUE;
78      END_IF;
79      SpdAct_Pps := DINT_TO_REAL(ABS(Encoder_Pos - EncPrev)) * 10.0;
80      EncPrev := Encoder_Pos;
81  END_IF;
82  RwSpeed := SpdAct_Pps * MmPerPulse * 0.06;          (* m/min *)
83
84  (* ---- 4. Learn material factor  K = dDiam/dPulse * Diam ----------------- *)
85  IF Tick THEN
86      LearnWinTicks := LearnWinTicks + 1;
87      IF NOT LearnInit THEN
88          LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiam;
89          LearnWinTicks := 0;
90          LearnWinRunning := O_MotorStart;
91          LearnInit := TRUE;
92      ELSE
93          LearnWinRunning := LearnWinRunning AND O_MotorStart;
94          IF LearnWinTicks >= RwWinTicks THEN
95              LearnWin_Pls := ABS(Encoder_Pos - LearnWinStartPos);
96              LearnWinDiamDelta := RwDiam - LearnWinStartDiam;
97              (* A window SHOULD learn: ran all the time, enough length,  *)
98              (* DT35 OK. Then it must give a plausible paper thickness. *)
99              IF LearnWinRunning AND (LearnWin_Pls >= RwMinPulses)
100               AND NOT RwSensFault THEN
101                 LearnWinOk := FALSE;
102                 IF (LearnWinDiamDelta > 0.0) AND (MmPerPulse > 0.0) THEN
103                     MatK_Window := (LearnWinDiamDelta / DINT_TO_REAL(LearnWin_Pls))
104                                 * ((RwDiam + LearnWinStartDiam) / 2.0);
105                     (* K = 2 * thickness * mm-per-pulse / PI          *)
106                     ThickWin_mm := MatK_Window * 3.14159 / (2.0 * MmPerPulse);
107                     IF (ThickWin_mm >= RwThickMin_mm)
108                       AND (ThickWin_mm <= RwThickMax_mm) THEN
109                         LearnWinOk := TRUE;
```

```
                IF RwMat_K <= 0.0 THEN
                    RwMat_K := MatK_Window;
                ELSE
                    RwMat_K := RwMat_K + RwLearn_K * (MatK_Window - RwMat_K);
                END_IF;
            END_IF;
        END_IF;
        IF LearnWinOk THEN
            LearnFailCnt := 0;
        ELSIF LearnFailCnt < 1000 THEN
            LearnFailCnt := LearnFailCnt + 1;  (* no growth / implausible *)
        END_IF;
    END_IF;
    LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiam;
    LearnWinTicks := 0;
    LearnWinRunning := O_MotorStart;
  END_IF;
END_IF;

(* ---- 5. Learn real stop deceleration on every stop ---------- *)
(* v6d: a run with jog / manual / slow button uses the 2nd ramp (O_AccDec2) *)
(*      -> its stop must not be learned. Latched for the whole run because  *)
(*      I_LowSpd_OS is already released on the stop edge.                   *)
IF O_MotorStart AND NOT MotorRunMem THEN          (* start edge *)
    RunWasJog := FALSE;
END_IF;
IF O_MotorStart AND (ManMode OR I_LowSpd_OS) THEN
    RunWasJog := TRUE;
END_IF;

IF MotorRunMem AND NOT O_MotorStart THEN          (* stop command edge *)
    StopMeasActive := TRUE;
    StopStartPos := Encoder_Pos;
    StopStartSpd_Pps := SpdAct_Pps;
    StandstillTicks := 0;
    StopLearnValid := ControlVoltage AND NOT RunWasJog AND NOT ManMode;
END_IF;
MotorRunMem := O_MotorStart;

IF O_MotorStart THEN
    StopMeasActive := FALSE;                  (* restarted: abort *)
ELSIF StopMeasActive AND Tick THEN
    IF NOT ControlVoltage THEN
        StopLearnValid := FALSE;              (* E-stop during the stop *)
    END_IF;
    IF SpdAct_Pps <= 20.0 THEN                (* <= 2 counts / 100 ms *)
        StandstillTicks := StandstillTicks + 1;
    ELSE
        StandstillTicks := 0;
    END_IF;
    IF StandstillTicks >= 3 THEN              (* standstill 300 ms *)
        StopMeasActive := FALSE;
        StopMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - StopStartPos));
        IF RwLearnStop AND StopLearnValid
          AND (StopStartSpd_Pps * MmPerPulse * 0.06 >= RwMinLearnSpd) THEN
          (* This stop SHOULD learn: result must be plausible       *)
          StopLearnOk := FALSE;
          IF StopMeas_Pls > StopStartSpd_Pps * RwTreact THEN
```

```
169             StopDecelMeas := (StopStartSpd_Pps * StopStartSpd_Pps)
170                     / (2.0 * (StopMeas_Pls - StopStartSpd_Pps * RwTreact));
171             StopDecelMeas_mm := StopDecelMeas * MmPerPulse;  (* mm/s2 *)
172             IF (StopDecelMeas_mm >= RwDecelMin_mm)
173               AND (StopDecelMeas_mm <= RwDecelMax_mm) THEN
174                StopLearnOk := TRUE;
175                IF RwStopDecel <= 0.0 THEN
176                   RwStopDecel := StopDecelMeas;
177                ELSE
178                   RwStopDecel := RwStopDecel + 0.3 * (StopDecelMeas -
RwStopDecel);
179                END_IF;
180             END_IF;
181          END_IF;
182          IF StopLearnOk THEN
183             StopFailCnt := 0;
184          ELSIF StopFailCnt < 1000 THEN
185             StopFailCnt := StopFailCnt + 1;
186          END_IF;
187       END_IF;
188    END_IF;
189 END_IF;
190
191 (* ---- 6. Lengths in pulses: stop, slow-down, remaining ------------------ *)
192 (* Stop length uses the ACTUAL speed (precision)                          *)
193 IF RwStopDecel > 0.0 THEN
194    StopLen_Pls := SpdAct_Pps * RwTreact
195                + (SpdAct_Pps * SpdAct_Pps) / (2.0 * RwStopDecel);
196 ELSE
197    StopLen_Pls := SpdAct_Pps * (RwTreact + RwTstopDef / 2.0);
198 END_IF;
199 StopLen_Pls := StopLen_Pls * RwTrim;
200
201 (* Slow-down uses SpdRef = MAX(actual, work-speed setpoint):              *)
202 (* stable value, shown to the operator even before start                  *)
203 SlowLenBase_Pls := 0.0;  RwSlowLen_Pls := 0.0;  UwSlowLen_Pls := 0.0;
204 IF MmPerPulse > 0.0 THEN
205    IF RwSlowDecel > 0.0 THEN
206       SlowDecel_Pps2 := RwSlowDecel / MmPerPulse;
207    ELSE
208       SlowDecel_Pps2 := DecelCoarse * 250.0 / MmPerPulse; (* 1 = 250 mm/s2 *)
209    END_IF;
210    (* v6d: DecelTrim ramp used inside the +/- SpdDelta_Percent band *)
211    IF DecelTrim > 0.0 THEN
212       TrimDecel_Pps2 := DecelTrim * 250.0 / MmPerPulse;
213    ELSE
214       TrimDecel_Pps2 := SlowDecel_Pps2;
215    END_IF;
216    SpdBand_Pps := SpdDelta_Percent * 1000.0 / 60.0 / MmPerPulse;
217
218    (* v6d: speeds from the potentiometers, valid in auto AND ManMode     *)
219    (*      (m/min = Pot / Input_Ratio, same as Motor_Manager rung 8)     *)
220    IF Input_Ratio > 0.0 THEN
221       SpdFinal_Pps := INT_TO_REAL(Pot2_Mem) / Input_Ratio * 1000.0 / 60.0 /
MmPerPulse;
222       SpdWork_Pps  := INT_TO_REAL(Pot1_Mem) / Input_Ratio * 1000.0 / 60.0 /
MmPerPulse;
223    ELSE
224       SpdFinal_Pps := LowSpd_Display * 1000.0 / 60.0 / MmPerPulse;
```

```
    SpdWork_Pps  := Speed_Display  * 1000.0 / 60.0 / MmPerPulse;
  END_IF;
  SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);

  IF RwStopDecel > 0.0 THEN
    StopLenFinal_Pls := SpdFinal_Pps * RwTreact
                + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * RwStopDecel);
  ELSE
    StopLenFinal_Pls := SpdFinal_Pps * (RwTreact + RwTstopDef / 2.0);
  END_IF;

  SlowLenBase_Pls := StopLenFinal_Pls * RwTrim;
  IF (SpdRef_Pps > SpdFinal_Pps) AND (SlowDecel_Pps2 > 0.0) THEN
    IF (RwSlowDecel > 0.0) OR (TrimDecel_Pps2 <= 0.0) THEN
      (* manual decel: one constant ramp *)
      SlowLenBase_Pls := SlowLenBase_Pls
        + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
         / (2.0 * SlowDecel_Pps2);
    ELSE
      (* auto: coarse ramp down to the band, trim ramp inside it *)
      SpdKnee_Pps := SpdFinal_Pps + SpdBand_Pps;
      IF SpdRef_Pps > SpdKnee_Pps THEN
        SlowLenBase_Pls := SlowLenBase_Pls
          + (SpdRef_Pps * SpdRef_Pps - SpdKnee_Pps * SpdKnee_Pps)
           / (2.0 * SlowDecel_Pps2)
          + (SpdKnee_Pps * SpdKnee_Pps - SpdFinal_Pps * SpdFinal_Pps)
           / (2.0 * TrimDecel_Pps2);
      ELSE
        SlowLenBase_Pls := SlowLenBase_Pls
          + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
           / (2.0 * TrimDecel_Pps2);
      END_IF;
    END_IF;
  END_IF;

  RwSlowLen_Pls := SlowLenBase_Pls + RwSlowMargin_m * 1000.0 / MmPerPulse;

  IF UwSlowMargin_m > 0.0 THEN
    UwMarginUsed_m := UwSlowMargin_m;
  ELSE
    UwMarginUsed_m := RwSlowMargin_m;
  END_IF;
  UwSlowLen_Pls := SlowLenBase_Pls + UwMarginUsed_m * 1000.0 / MmPerPulse;
END_IF;

IF RwDiam >= RwTargetDiam THEN
  RwRemain_Pls := 0.0;
ELSIF RwMat_K > 0.0 THEN
  RwRemain_Pls := (RwTargetDiam * RwTargetDiam - RwDiam * RwDiam)
            / (2.0 * RwMat_K);
ELSE
  RwRemain_Pls := 1.0E9;                    (* K not learned: plain target *)
END_IF;

RwRemain_m  := RwRemain_Pls  * MmPerPulse / 1000.0;
RwStopLen_m := StopLen_Pls   * MmPerPulse / 1000.0;
RwSlowLen_m := RwSlowLen_Pls * MmPerPulse / 1000.0;

(* ---- 6b. Live operator displays ---------------------------------------- *)
```

```
(* Rewinder: diameter where slow-down / stop command will happen          *)
IF RwMat_K > 0.0 THEN
    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
    IF SqrtArg > 0.0 THEN
        RwSlowAtDiam := SQRT(SqrtArg);
    ELSE
        RwSlowAtDiam := 0.0;
    END_IF;
    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * StopLen_Pls;
    IF SqrtArg > 0.0 THEN
        RwStopAtDiam := SQRT(SqrtArg);
    ELSE
        RwStopAtDiam := 0.0;
    END_IF;
ELSE
    RwSlowAtDiam := RwTargetDiam;                (* not learned yet *)
    RwStopAtDiam := RwTargetDiam;
END_IF;

(* Unwinder: meter count where the slow-down will start                   *)
IF MmPerPulse > 0.0 THEN
    UwSlow_at_Metre := UwStop_At_Meter - UwSlowLen_Pls * MmPerPulse / 1000.0;
ELSE
    UwSlow_at_Metre := UwStop_At_Meter;
END_IF;
IF UwSlow_at_Metre < 0.0 THEN
    UwSlow_at_Metre := 0.0;
END_IF;

(* Unwinder slow-down request, latched                                    *)
IF (Current_Metre < UwSlow_at_Metre - 50.0) OR (UwStop_At_Meter <= 0.0) THEN
    UwSlowReq := FALSE;                          (* counter reset / longer roll *)
END_IF;
IF (UwStop_At_Meter > 0.0) AND (Current_Metre >= UwSlow_at_Metre) THEN
    UwSlowReq := TRUE;
END_IF;

(* ---- 7. Rewinder requests: latched, reset when a new core is loaded ---- *)
IF NOT RwStopEnable
   OR ((RwDiam < RwTargetDiam - RwResetHyst) AND NOT RwSensFault) THEN
    RwStopReq := FALSE;
    RwSlowReq := FALSE;
END_IF;

IF RwStopEnable THEN
    IF (RwRemain_Pls <= StopLen_Pls) OR (RwSensFault AND RwStopOnFault) THEN
        RwStopReq := TRUE;
    END_IF;
    IF RwUseSlow AND (RwRemain_Pls <= RwSlowLen_Pls) THEN
        RwSlowReq := TRUE;
    END_IF;
END_IF;

(* ---- 8. One AUTO at a time + single "SLOW DOWN AT" display ------------- *)
IF UwStopEnable AND NOT UwEnMem THEN             (* unwinder AUTO switched
ON *)
    RwStopEnable := FALSE;
END_IF;
```

```
    IF RwStopEnable AND NOT RwEnMem THEN                (* rewinder AUTO switched
    ON *)
        UwStopEnable := FALSE;
    END_IF;
    UwEnMem := UwStopEnable;
    RwEnMem := RwStopEnable;

    IF RwStopEnable THEN
        SlowAtDisp := RwSlowAtDiam;                 (* mm *)
        SlowAtMode := 2;
    ELSIF UwStopEnable THEN
        SlowAtDisp := UwSlow_at_Metre;              (* m  *)
        SlowAtMode := 1;
    ELSE
        SlowAtDisp := 0.0;
        SlowAtMode := 0;                        (* hide display *)
    END_IF;
    SlowAtShow := (SlowAtMode <> 0);

    (* ---- 9. Automatic stop master = selector 12SA5 (I_AutoStop_OS) -------- *)
    AutoSlowCmd := I_AutoStop_OS AND
                   ((UwStopEnable AND UwSlowReq) OR (RwStopEnable AND RwSlowReq));

    AutoStopCmd := I_AutoStop_OS AND
                   ((UwStopEnable AND (UwStop_At_Meter > 0.0)
                     AND (Current_Metre >= UwStop_At_Meter))
                    OR (RwStopEnable AND RwStopReq));

    (* ---- 10. Unwinder roll empty (for alarm 15) ---------------------------- *)
    IF UwDiam_Act > (UwDiamStop + UwEmptyHyst) THEN
        UnWinder_Empty := FALSE;                    (* new roll loaded *)
    END_IF;
    IF (UwDiam_Act > 0.0) AND (UwDiam_Act <= UwDiamStop) THEN
        UnWinder_Empty := TRUE;                     (* roll empty *)
    END_IF;

    (* ---- 11. Rewinder max diameter protection - ALWAYS active ------------- *)
    (* Independent of selector 12SA5 and of the AUTO buttons.               *)
    (* Mechanical limit 680 mm radius = 1360 mm                             *)
    IF NOT RwSensFault THEN
        RwLastValidDiam := RwDiam;
    END_IF;

    IF RwDiam >= RwMaxDiam THEN
        RwRemainMax_Pls := 0.0;
    ELSIF RwMat_K > 0.0 THEN
        RwRemainMax_Pls := (RwMaxDiam * RwMaxDiam - RwDiam * RwDiam)
                           / (2.0 * RwMat_K);
    ELSE
        RwRemainMax_Pls := 1.0E9;                   (* K not learned: plain limit *)
    END_IF;

    (* Display: diameter where the max-protection slow-down will start      *)
    IF RwMat_K > 0.0 THEN
        SqrtArg := RwMaxDiam * RwMaxDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
        IF SqrtArg > 0.0 THEN
            RwMaxSlowAtDiam := SQRT(SqrtArg);
        ELSE
            RwMaxSlowAtDiam := 0.0;
```

```
399     END_IF;
400   ELSIF RwMaxSlowFallback > 0.0 THEN
401     RwMaxSlowAtDiam := RwMaxSlowFallback;          (* fallback in use *)
402   ELSE
403     RwMaxSlowAtDiam := RwMaxDiam;
404   END_IF;
405
406 (* Reset only after the roll is removed (diameter well below the limit)    *)
407 IF (RwDiam < RwMaxDiam - RwResetHyst) AND NOT RwSensFault THEN
408     RwMaxStop := FALSE;
409     RwMaxSlow := FALSE;
410   END_IF;
411
412 IF NOT RwSensFault THEN
413     IF RwRemainMax_Pls <= StopLen_Pls THEN
414       RwMaxStop := TRUE;                    (* speed-compensated stop *)
415     END_IF;
416     IF RwRemainMax_Pls <= RwSlowLen_Pls THEN
417       RwMaxSlow := TRUE;                    (* warning slow-down + horn *)
418     END_IF;
419     IF (RwMat_K <= 0.0) AND (RwMaxSlowFallback > 0.0)
420       AND (RwDiam >= RwMaxSlowFallback) THEN
421       RwMaxSlow := TRUE;                    (* fallback while K not learned *)
422     END_IF;
423   ELSIF (AI_Diam_Rewinder < RwRawMin) AND (RwLastValidDiam > RwMaxDiam -
    100.0) THEN
424     RwMaxStop := TRUE;                    (* roll grew into the DT35 near zone *)
425   END_IF;
426
427 (* Max diameter protection added OUTSIDE the 12SA5 master (section 9)      *)
428   AutoSlowCmd := AutoSlowCmd OR RwMaxSlow;
429   AutoStopCmd := AutoStopCmd OR RwMaxStop;
430
431 (* ---- 12. Learning supervision -> alarm 16 (section Alarms) ---------- *)
432 (* Learned paper thickness, for display / plausibility check              *)
433 IF MmPerPulse > 0.0 THEN
434     RwThick_mm := RwMat_K * 3.14159 / (2.0 * MmPerPulse);
435   ELSE
436     RwThick_mm := 0.0;
437   END_IF;
438
439 (* Fault: too many consecutive windows / stops that should have           *)
440 (* learned but did not. Clears by itself on the next good result.        *)
441   RwLearnFault := (LearnFailCnt >= RwLearnFailMax)
442                 OR (StopFailCnt >= RwStopFailMax);
```

1-8.Data

1-8-1.Data Types

Name	Type	Comment

1-8-2.Global Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Network Publish	Comment
VAR_GLOBAL							
Pot1_Work_Velocity	INT		BuiltInIO://opt/#0/Ch1_Analog_Input_Value	False	False	Publish Only	Potentiometer 1 value - Work speed -
InvRef	INT		BuiltInIO://opt/#0/Ch2_Analog_Output_Value	False	False	Publish Only	Inverter speed reference
Fix10Vdc	INT		BuiltInIO://opt/#0/Ch1_Analog_Output_Value	False	False	Publish Only	Fixed 10 Vdc generator
VelMax	REAL	800		True	False	Publish Only	Maximum speed
VelSet	REAL			False	False	Publish Only	Set speed
RwMaxSlowFallback	REAL	1250.0		True	False	Publish Only	Rewinder max: slow-down diameter while K not learned (mm)
RwThickMin_mm	REAL	0.02		True	False	Publish Only	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL	2.0		True	False	Publish Only	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL	20.0		True	False	Publish Only	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL	5000.0		True	False	Publish Only	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT	12		True	False	Publish Only	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT	3		True	False	Publish Only	Learning: failed stops before alarm
RwThick_mm	REAL			False	False	Publish Only	Learned paper thickness (mm)
OverSpeed	BOOL			False	False	Publish Only	Speed too high
UnderSpeed	BOOL			False	False	Publish Only	Speed too low
SpdDelta_Percent	REAL			True	False	Publish Only	Speed difference percentage
Encoder_Pos	DINT		IOBus://unit#3/Ch1 Encoder Present Position	False	False	Publish Only	Encoder counter
PulseRate	UDINT		IOBus://unit#3/Ch1 Pulse Rate	False	False	Publish Only	Pulses per 100 ms
Encoder_On	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Counter Enable	False	False	Publish Only	Encoder enable
Encoder_Reset	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Internal R	False	False	Publish Only	Encoder reset

			eset Execution				
HMI_Screen	ARRAY[0..1] OF DINT			False	False	Publish Only	HMI panel Screen
Velocity_M_min	REAL			True	False	Publish Only	Speed in m/min
Current_Metre	REAL			True	False	Publish Only	Current metres
I_RsetCnt	BOOL			False	False	Publish Only	Metre counter reset button
Speed_Display	REAL			True	False	Publish Only	Set speed (displayed)
I_ControlVoltage	BOOL		BuiltInIO://cpu/#0/Input_Bit_00	False	False	Publish Only	Auxiliaries ON
I_Start_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_01	False	False	Publish Only	Start button
I_Stop_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_02	False	False	Publish Only	Stop button
I_Motor_Thermal	BOOL		BuiltInIO://cpu/#0/Input_Bit_03	False	False	Publish Only	Motor thermal switch
I_Prot_ByPass	BOOL		BuiltInIO://cpu/#0/Input_Bit_04	False	False	Publish Only	Guards-bypassed selector switch - Under Panel
I_Hub_Position	BOOL		BuiltInIO://cpu/#0/Input_Bit_05	False	False	Publish Only	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL		BuiltInIO://cpu/#0/Input_Bit_06	False	False	Publish Only	Front guard closed
I_LowSpd_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_07	False	False	Publish Only	Slow speed button
I_VFD_Alarm	BOOL		BuiltInIO://cpu/#0/Input_Bit_08	False	False	Publish Only	Inverter in alarm
I_SafeGuard_Closed	BOOL			False	False	Publish Only	Side doors closed
I_OverRider_PH	BOOL		BuiltInIO://cpu/#0/Input_Bit_13	False	False	Publish Only	Roll diameter photocell
I_OR_Position_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN01 (Pin 1.1) _6000_01	False	False	Publish Only	OverRider position sens
I_OR_Pressure_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN02 (Pin 2.1) _6000_02	False	False	Publish Only	OverRider pressure switch
I_OR_Auto_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN03 (Pin 1.	False	False	Publish Only	OverRider in automatic

			2)_6000_03				
I_OR_Cont_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN04 (Pin 2.2) _6000_04	False	False	Publish Only	OverRider in continuous
O_MotorStart	BOOL		BuiltInIO://cpu/#0/Output_Bit_00	False	False	Publish Only	Motor start
I_FanTemp_Ok	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3) _6000_06	False	False	Publish Only	Motor (fan) thermal OK
O_Alarm_Array	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 01 (Pin 1.1) _7000_01	False	False	Publish Only	Alarm lamp
O_Prot_ByPass_Lamp	BOOL			False	False	Publish Only	Guards bypassed lamp
O_Motor_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_01	False	False	Publish Only	Motor ON lamp
O_RwUnload	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 03 (Pin 1.2) _7000_03	False	False	Publish Only	Unloaders enable - opening
O_FrontGuard_ByPass	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 04 (Pin 2.2) _7000_04	False	False	Publish Only	Front guard bypass
O_Brake	BOOL		BuiltInIO://cpu/#0/Output_Bit_02	False	False	Publish Only	Brake command
O_OR_Auto_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_06	False	False	Publish Only	OverRider auto/continuous lamp
O_OverRider_Up	BOOL		BuiltInIO://cpu/#0/Output_Bit_09	False	False	Publish Only	Raise OverRider ~10mm Over Rewinder roll
O_OverRider_Hold	BOOL		BuiltInIO://cpu/#0/Output_Bit_04	False	False	Publish Only	OverRider Lock
O_CoolingFan	BOOL		BuiltInIO://cpu/#0/Output_Bit_03	False	False	Publish Only	Brake and motor fans
Fan_Timeout	REAL	0.5		True	False	Publish Only	Fans stop timeout [min]
OverRider_TimeOFF	REAL	0.01		True	False	Publish Only	OverRider photocell OFF timeout (panel)
OverRider_Ti	REAL	0.05		True	False	Publish Only	OverRider photocell ON ti

meON							meout (panel)
AlarmOn	BOOL			False	False	Publish Only	Alarm active
BuzzerOn	BOOL			False	False	Publish Only	Panel buzzer command
Msg	INT			False	False	Publish Only	Message
Index_B	INT			False	False	Publish Only	Alarm index
MotStop	BOOL			False	False	Publish Only	Motor stopped
Run_Normal_Spd	BOOL			False	False	Publish Only	Motor running - normal speed
Clock1s	BOOL			False	False	Publish Only	1 s clock
ManMode	BOOL			True	False	Publish Only	Manual mode
Speed_Man_SP	INT			True	False	Publish Only	% speed in manual
Counter_Reset	BOOL			False	False	Publish Only	Counter reset from panel
I_FrontGuard_Open	BOOL		BuiltInIO://cpu/#0/Input_Bit_09	False	False	Publish Only	Front guard open
RwUnload_Enable	BOOL		BuiltInIO://cpu/#0/Output_Bit_07	False	False	Publish Only	Unloaders enable - closing
JogSpd_SP	REAL	10.0		True	False	Publish Only	Speed in manual
O_AccDec2	BOOL		BuiltInIO://cpu/#0/Output_Bit_05	False	False	Publish Only	Second acc/dec ramp
AccelTrim	REAL			True	False	Publish Only	Acceleration coefficient in run
DecelTrim	REAL			True	False	Publish Only	Deceleration coefficient in run
DecelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests accel)
O_AcousticSignal	BOOL		BuiltInIO://cpu/#0/Output_Bit_08	False	False	Publish Only	Horn / Acoustic signal
Pot2_Slow_Velocity	INT		BuiltInIO://opt/#0/Ch2_Analog_Input_Value	False	False	Publish Only	Potentiometer 2 value - Final speed -
I_AutoStop_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_10	False	False	Publish Only	Stop on metre counter
I_RollEnd_SlowDown	BOOL		BuiltInIO://cpu/#0/Input_Bit_11	False	False	Publish Only	Slow down at end of roll
I_WebBreak_Monitor	BOOL		BuiltInIO://cpu/#0/Input_Bit_12	False	False	Publish Only	Paper break monitoring ON
I_Vacuum_OK	BOOL		IOBus://unit#3/Ch1 Reset/External Input Status/Ch1 External Input 0 Status	False	False	Publish Only	Extractor fan thermal OK
I_SlowSens_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_I	False	False	Publish Only	Slow-down photocell

			N05 (Pin 1.3) _6000_05				
I_WebBreak_PH	BOOL		ECAT://node #[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN07 (Pin 1.4) _6000_07	False	False	Publish Only	Paper break photocell
UwStop_At_Meter	REAL	5100		True	False	Publish Only	Unwinder Stop at metres
UwSlowMargin_m	REAL	10		True	False	Publish Only	Unwinder Slow-down metres
LowSpd_Display	REAL			True	False	Publish Only	Slow-down speed (displayed)
RwMaxDiam	REAL	1330.0		True	False	Publish Only	Rewinder max diameter protection (mm)
RwMaxStop	BOOL			False	False	Publish Only	Rewinder max diameter STOP (latched)
Encoder_Pulse	INT	1000		True	False	Publish Only	Encoder pulses/Rotation
LowSpd_Man_SP	INT			False	False	Publish Only	% slow-down speed in manual
CountReset_Req	BOOL			False	False	Publish Only	Counter reset requested
Psw	INT			True	False	Publish Only	Password
Language	DINT			True	False	Publish Only	Language
I_RsetAlarm	BOOL		ECAT://node #[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN08 (Pin 2.4) _6000_08	False	False	Publish Only	Alarm reset
Fix10Vdc_1	UINT			False	False	Publish Only	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL	400		True	False	Publish Only	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL			False	False	Publish Only	Panel alarms
SafeGuard_ByPass	BOOL			False	False	Publish Only	Guards bypassed
FrontGuard_Closed	BOOL			False	False	Publish Only	Front guard closed
Anal3	UINT			False	False	Publish Only	(?) Analog 3
Anal4	UINT			False	False	Publish Only	(?) Analog 4
UwDiam_Measure	REAL			True	False	Publish Only	Measured distance
UwCoreDiameter	REAL	100		True	False	Publish Only	Core Outside diameter [mm]
Core_Measure	BOOL	False		True	False	Publish Only	Stored core value
UwCoreDistance	REAL	785.56		True	False	Publish Only	Sonar distance to empty core. [mm]
AI_Diam_Unwinder	INT		IOBus://unit#2/Ch1 Analog Input Value	False	False	Publish Only	Analog value - UnWinder Diameter
Measure_ON	BOOL	True		True	False	Publish Only	Measurement enabled
UwDiam_Act	REAL			True	False	Publish Only	Measured roll diameter
UwSlowDown_Diam	REAL	200		True	False	Publish Only	Unwinder Slow-down diameter [mm]

UwDiamStop	REAL	110		True	False	Publish Only	Stop diameter
Alarm_Reset	BOOL			False	False	Publish Only	Alarm reset from panel
Psw22	BOOL			False	False	Publish Only	(?) Password 22
Psw44	BOOL			False	False	Publish Only	(?) Password 44
I_Roll_OK	BOOL			False	False	Publish Only	Roll OK
I_RollEnd_PH	BOOL			False	False	Publish Only	End-of-roll photocell
AI_Diam_Rewinder	INT		IOBus://unit#2/Ch2 Analog Input Value	False	False	Publish Only	Analog value - ReWinder Diameter
RwStopEnable	BOOL	False		True	False	Publish Only	Rewinder diameter stop enabled
RwTargetDiam	REAL	300.0		True	False	Publish Only	Rewinder final diameter (mm)
RwUseSlow	BOOL	True		True	False	Publish Only	Slow down before target
RwSlowMargin_m	REAL	10.0		True	False	Publish Only	Rewinder Metres at slow speed before stop
RwSlowDecel	REAL	0.0		True	False	Publish Only	Rewinder Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL	100.0		True	False	Publish Only	Reset requests below target - this [mm]
RwDist0V	REAL	0.0		True	False	Publish Only	DT35 distance at 0 V (mm)
RwDist10V	REAL	750.0		True	False	Publish Only	DT35 distance at 10 V (mm)
RwRawMin	INT	10		True	False	Publish Only	Raw below = sensor fault
RwRawMax	INT	3900		True	False	Publish Only	Raw above = sensor fault
RwStopOnFault	BOOL	True		False	False	Publish Only	Stop on sensor fault
RwAxisDist	REAL	675.3		True	False	Publish Only	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL	0.2		True	False	Publish Only	Distance filter per 100 ms (0..1)
RwWinTicks	INT	100		True	False	Publish Only	Material learn window (100 = 10 s)
RwMinPulses	DINT	1000		True	False	Publish Only	Min pulses in window to learn
RwLearn_K	REAL	0.3		True	False	Publish Only	Material factor filter weight
RwMat_K	REAL	0.0		True	False	Publish Only	Learned material factor
RwLearnStop	BOOL	True		True	False	Publish Only	Learn stop decel on every stop
RwMinLearnSpd	REAL	50.0		True	False	Publish Only	Min speed to learn a stop (m/min)
RwStopDecel	REAL	0.0		True	False	Publish Only	Learned stop decel (pulses/s2)
RwTreact	REAL	0.1		True	False	Publish Only	Stop reaction time (s)
RwTstopDef	REAL	3.0		True	False	Publish Only	Stop time until decel learned (s)
RwTrim	REAL	1.0		True	False	Publish Only	Stop length trim (>1 = earlier)
RwDist	REAL			False	False	Publish Only	Filtered distance (mm)
RwDiam	REAL			False	False	Publish Only	Rewinder diameter (mm)
RwSpeed	REAL			False	False	Publish Only	Line speed from encoder (m/min)
RwRemain_m	REAL			False	False	Publish Only	Metres to target diameter
RwStopLen_m	REAL			False	False	Publish Only	Stopping length now (m)
RwSlowLen_m	REAL			False	False	Publish Only	Rewinder Slow-down length [m]

RwStopReq	BOOL			False	False	Publish Only	Rewinder diameter STOP request
RwSlowReq	BOOL			False	False	Publish Only	Rewinder diameter SLOW request
RwSensFault	BOOL			False	False	Publish Only	Rewinder DT35 sensor fault
UwStopEnable	BOOL	False		True	False	Publish Only	Unwnder Auto Stop - Enable
SlowAtDisp	REAL			False	False	Publish Only	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT			False	False	Publish Only	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL			False	False	Publish Only	HMI: show slow-down display
UwSlow_at_Metre	REAL			False	False	Publish Only	Unwinder: slow-down starts at [m]
UwSlowReq	BOOL			False	False	Publish Only	Unwinder metre slow-down request (latched)
RwSlowAtDiam	REAL			False	False	Publish Only	Rewinder slow-down starts at diameter [mm]
RwStopAtDiam	REAL			False	False	Publish Only	Rewinder: stop command at diameter (mm)
RwMaxSlow	BOOL			False	False	Publish Only	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL	5.0		True	False	Publish Only	Unwinder Empty Hysteresis
RwLearnFault	BOOL			False	False	Publish Only	Rewinder learning fault (alarm 16)
RwMaxSlowAtDiam	REAL			False	False	Publish Only	Rewinder max: slow-down starts at diameter (mm)